

多旋翼飞行器设计与控制
系列实验

RflySim高级版 第3讲 单机控制接口实现

戴训华 daixunhua@163.com

中南大学 计算机学院 副教授

北京航空航天大学 自动化学院 博士



北航可靠飞行控制研究组

BUAA Reliable Flight Control Group

大纲

1.软硬件配置

2.MAVLink通信解析

3.PX4官方控制器通信

4.代码生成控制器通信

5.小结

本讲的例程源码路径：
RflySimAPIs\SimulinkControlAPI

1.软硬件配置

注意：本小节例子需要使用PX4的官方固件，任意版本都可以，这里选择了最新固件

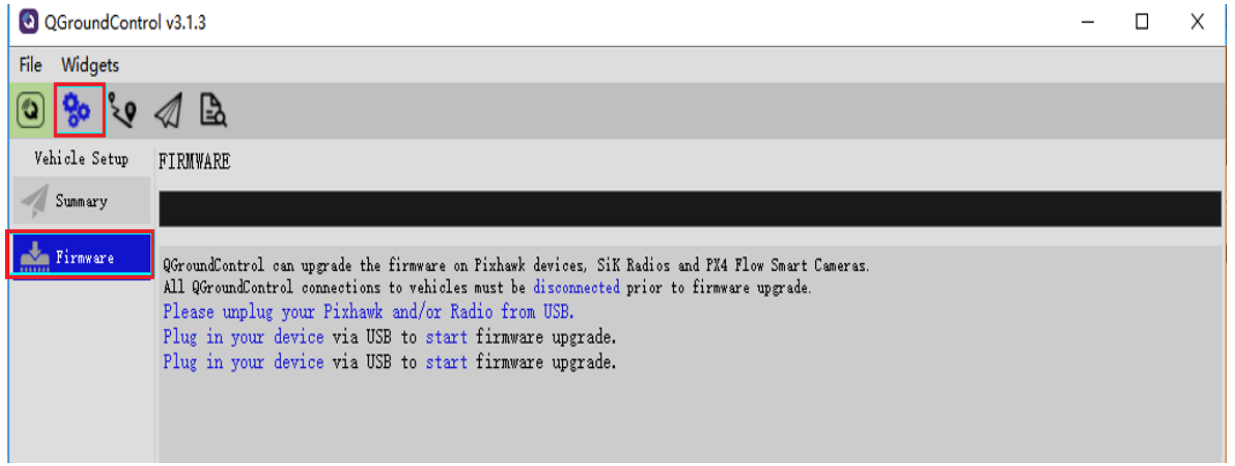
1.1 Pixhawk硬件配置(1) — 在线固件下载

方法如下：

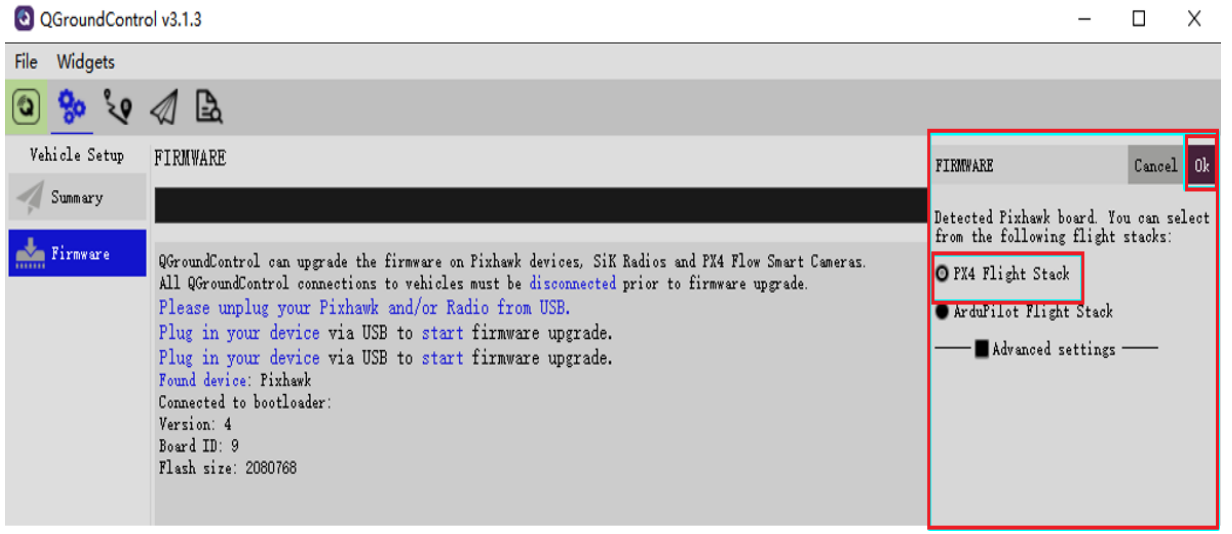
1）打开QGC地面站软件；

2）如右图上所示，点击工具栏齿轮图标进入设置页面，再点击“Firmware”标签进入固件 烧录页面；

3）用 USB 线连接 Pixhawk 自驾仪与电脑，此时软件会自动识别 Pixhawk 硬件，如右下图所示所示，在界面右侧弹出固件配置窗口，勾选“PX4 Flight Stack”，然后点击“OK”，QGC 开始自动下载（需联网，无法联网请参考下一讲使用本地固件）并安装最新的 PX4 固件到 Pixhawk中；



(a) 设置页面



(b) 固件烧录页面

北航可靠飞
BUAA Reliable Flight Control Group

1.软硬件配置

注意：如果文件夹中没有需要版本固件，请访问 https://github.com/PX4/Firmware/releases 下载

1.2 Pixhawk硬件配置(2) — 离线固件下载（因网络无法在线下载固件时采用）

方法如下：

1）打开QGC地面站软件；

2）点击“Firmware”标签，此时用USB数据线连接Pixhawk自驾仪，地面站会自动检测自驾仪

3）勾选“advanced settings”选择框；

4）点击“Standard Version (stable)”标签；

5）在弹出的文件选择页面中，如果使用Pixhawk1飞控选择“RflySimAPIs\FlightControlExpCourse\code\e0\4.PX4Firmwares\px4fmu-v3_default1.10.1Stable.px4”，如果使用Pixhawk 4，选择fmu-v5的固件

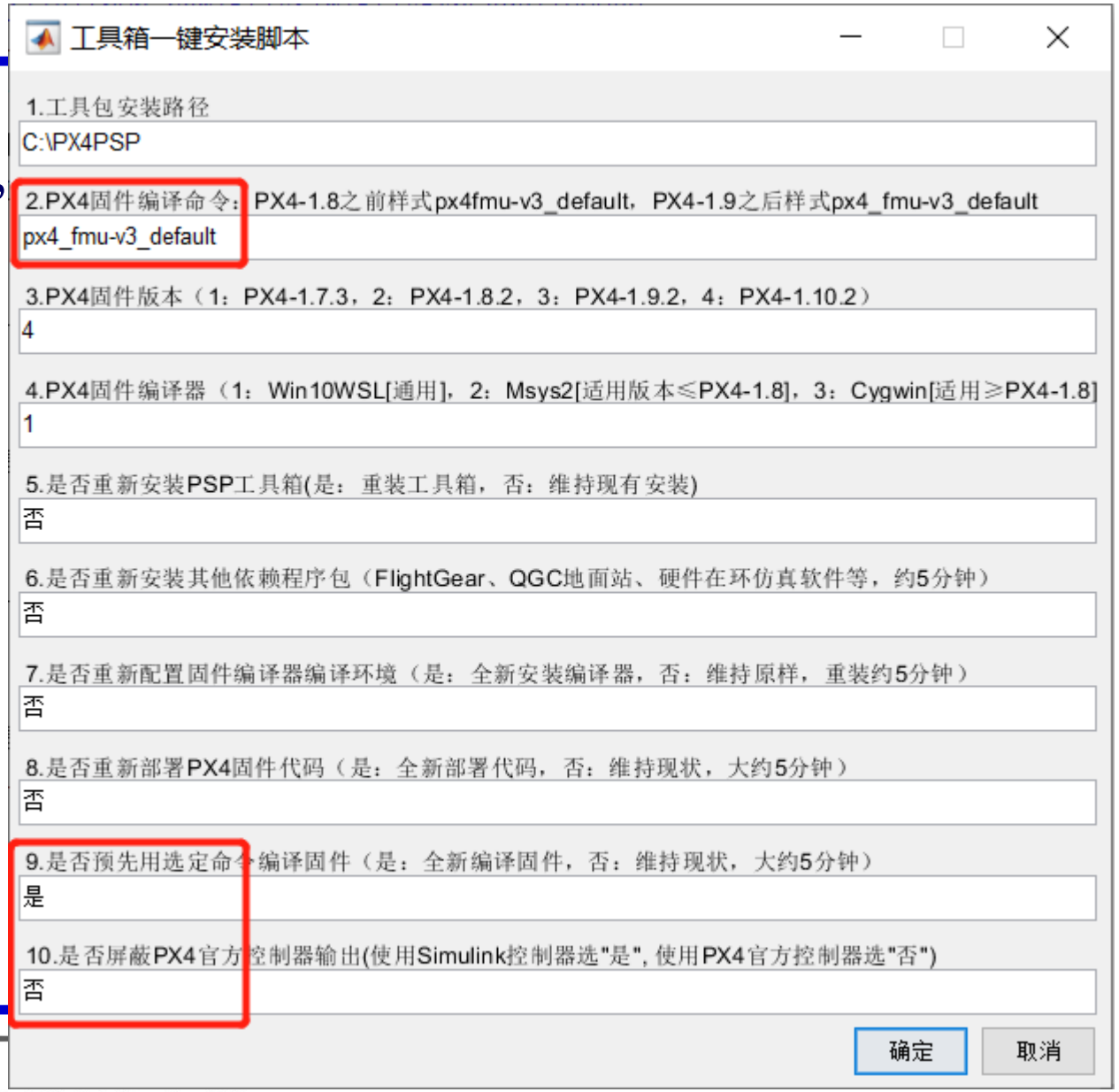


1.软硬件配置

注意：前两种方法都失效时才尝试本方法；下图针对RflySim高级版，如果是RflySim基础版请使用默认的固件和编译器版本。

1.3 Pixhawk硬件配置(3) — 离线固件生成

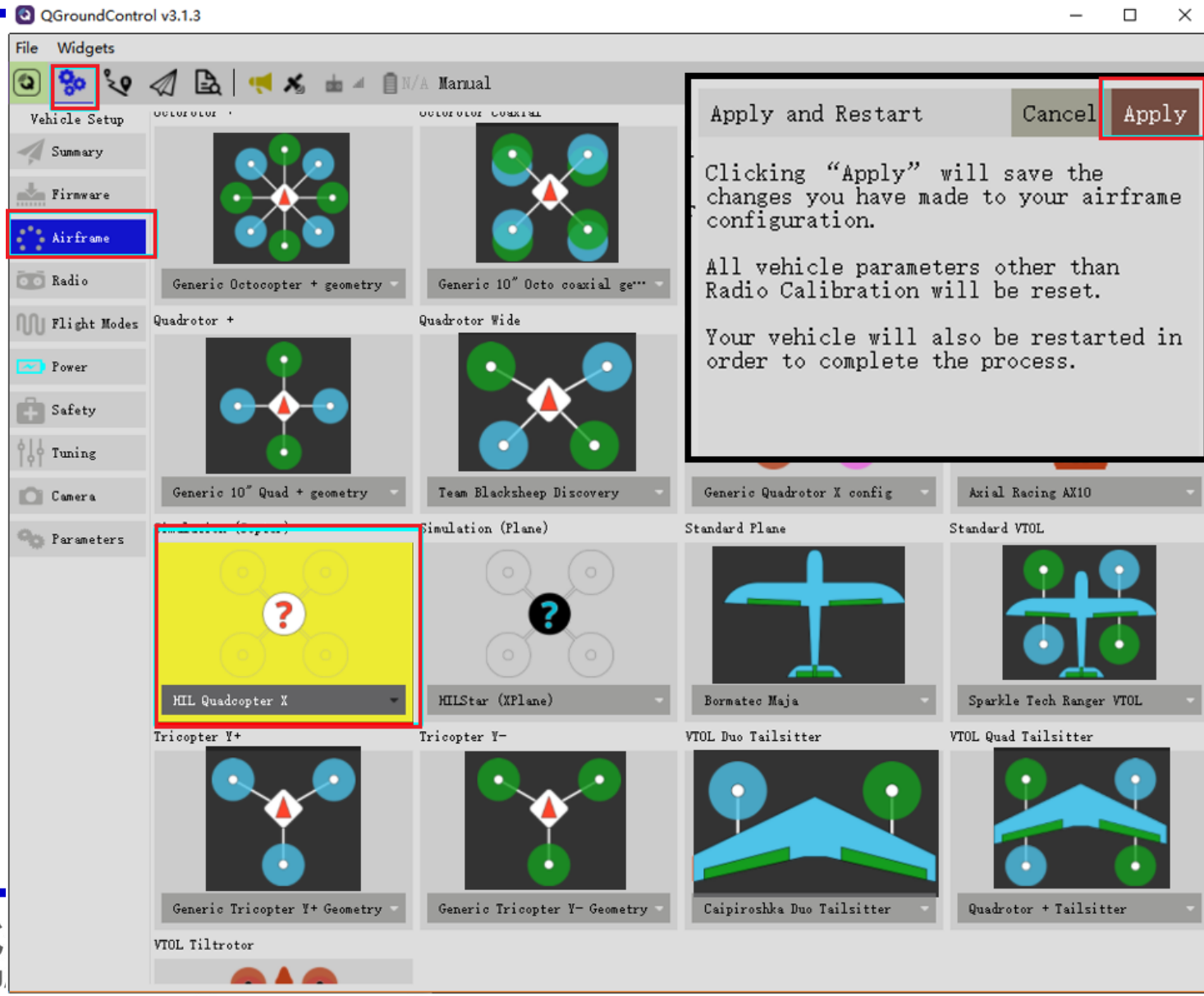
- 在MATLAB中重新运“OnekeyScript.p”脚本；
- 第2行输入“px4_fmu-v3_default”(这里针对Pixhawk 1，请根据自己的飞控硬件选择编译命令)
- 如右图所示，将第9项和第10项分别设置为“是”和“否”，其他选项保持默认，点击“确定”按钮，即可编译出未屏蔽PX4自身输出的PX4官方固件。
- 在MATLAB中运行“PX4Upload”命令，即可弹出固件烧录页面，此时插入Pixhawk飞控，即可将官方固件烧入。



1.软硬件配置

1.4 Pixhawk硬件在环仿真模式

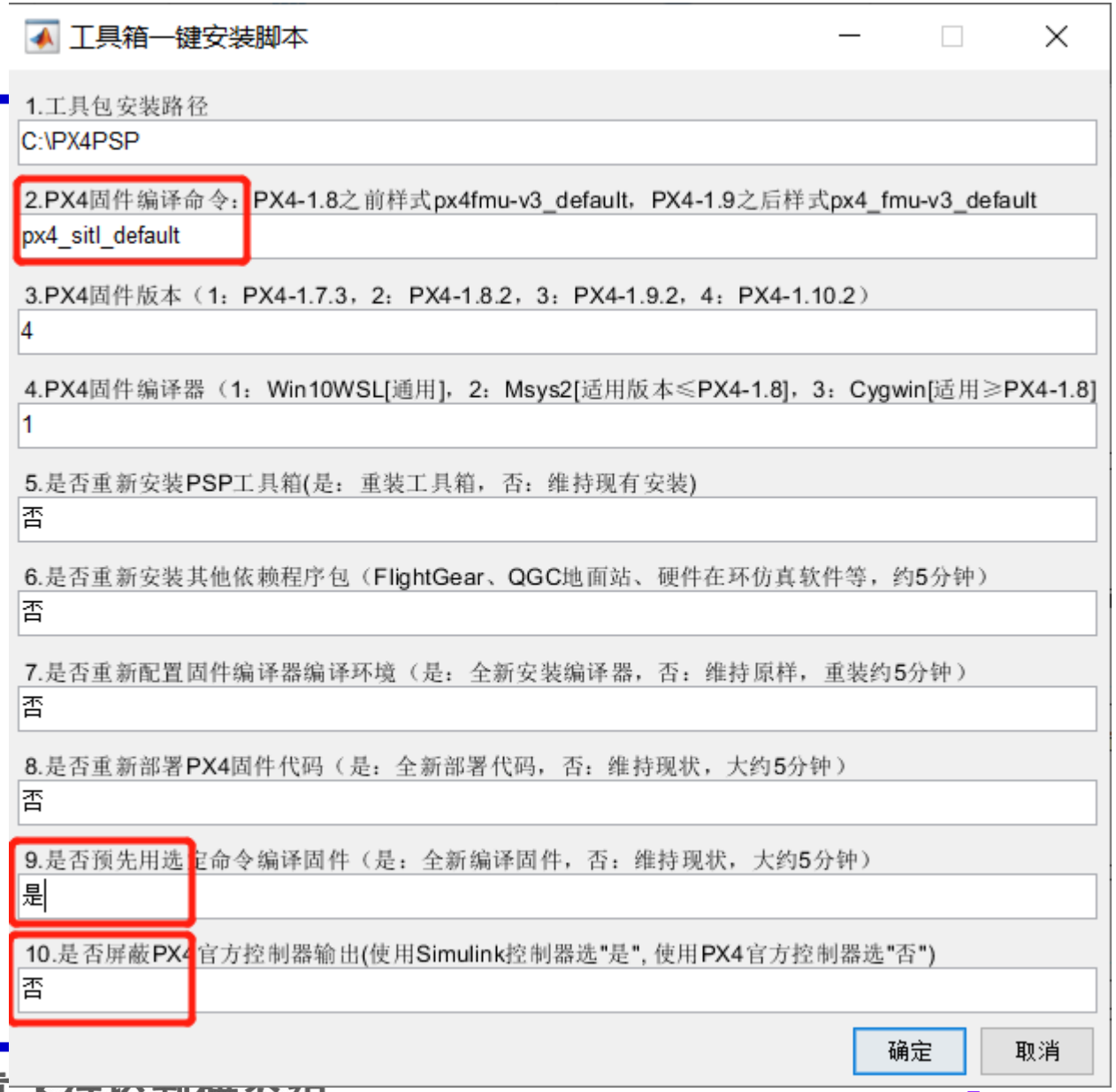
- 完成固件烧录后，自驾仪会自动重启并连接到QGC上；此时，如右图所示，进入“Airframe”标签页，选择机架类型为“HIL Quadcopter X”，然后点击右上角的“Apply and Restart”按钮，此时自驾仪会自动重启；
- 重启后QGC会自动寻找串口并连接到Pixhawk，此时查看各个配置页，确保Pixhawk进入硬件在环仿真模式。
- 完成遥控器校准和模式设置后，拔掉Pixhawk上的接收机，本讲课程不需要连接接收机



1.软硬件配置

1.5 PX4 SITL配置（仅限RflySim高级版）

- 重新运行“OnekeyScript.p”脚本，按照右图所示配置，点击“确定”
- 本配置是用于运行PX4的SITL软件在环仿真模式，使得我们可以在Windows下运行完整的PX4控制器，从而不需要Pixhawk硬件就能进行仿真。关键配置如下：
- 编译命令：“px4_sitl_default”
- 固件版本：“4”— PX4 1.10.2
- 编译器：“1”— Win10WSL
- 是否预编译固件：“是”
- 是否屏蔽PX4控制输出：“否”— 这里直接用PX4官方固件来做顶层控制

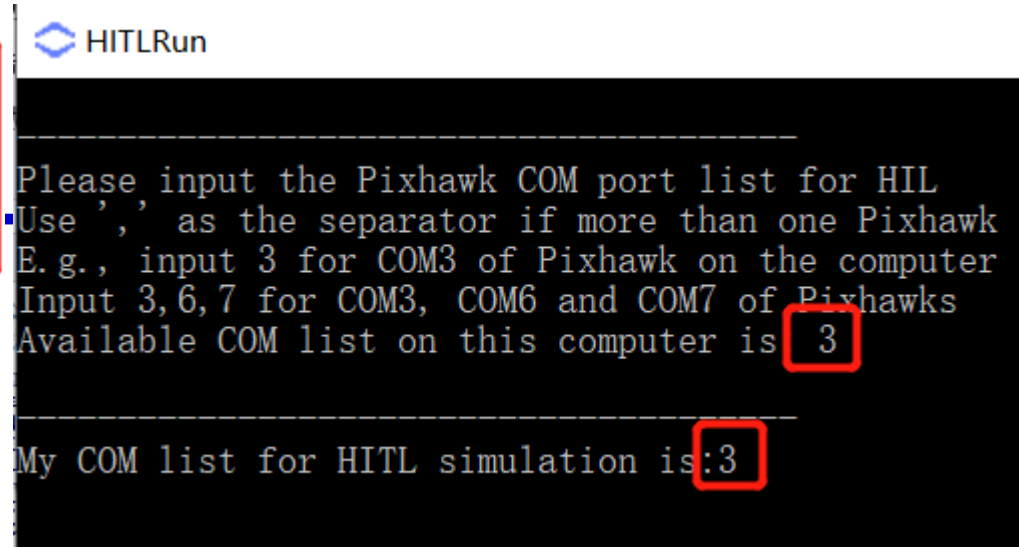


1.软硬件配置

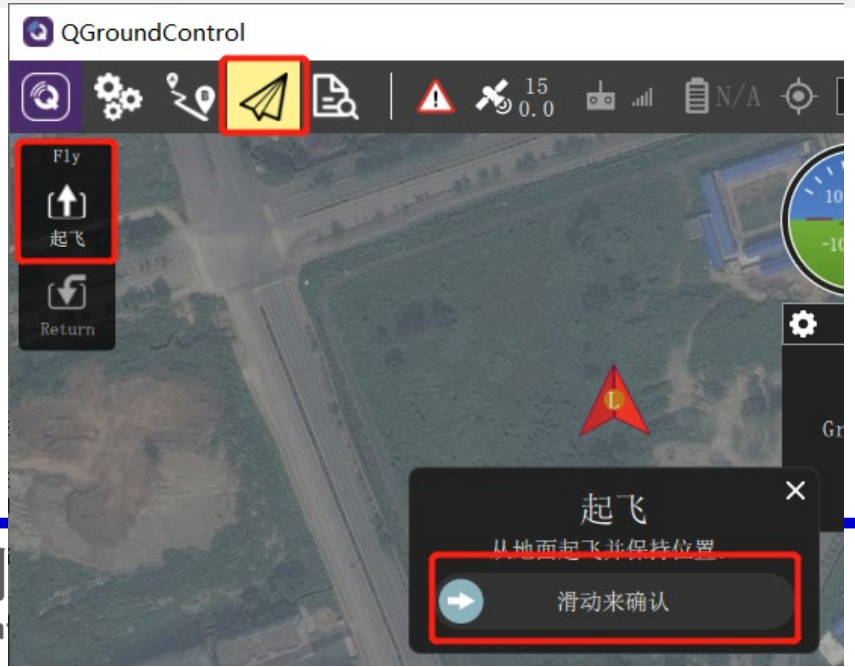
1.6 PX4 HITL硬件在环仿真测试

- 如果使用RflySim高级版，请插入Pixhawk，再直接运行桌面的HITLRun快捷方式，在弹出窗口中输入串口号，按下回车来开启硬件在环仿真系统
- 如果使用RflySim基础版，请插入Pixhawk，打开CopterSim软件，在“飞控选择”下拉框选择飞控串口，并点击“开始仿真”，然后手动打开QGC和3DDisplay
- 在QGC中，点击“纸飞机”按钮-起飞-滑动来确认，可以看到视景中飞机自动起飞，说明配置正确







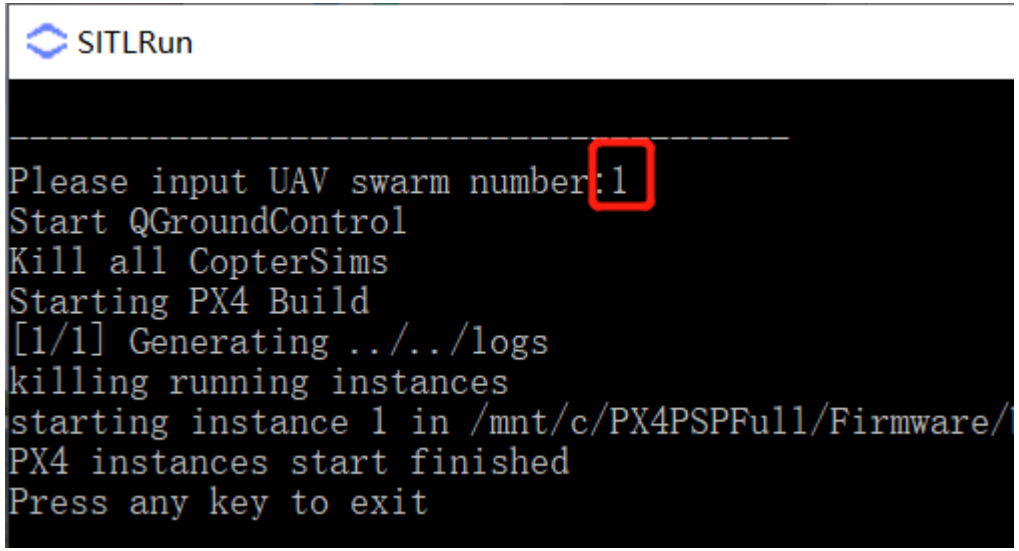


1.软硬件配置



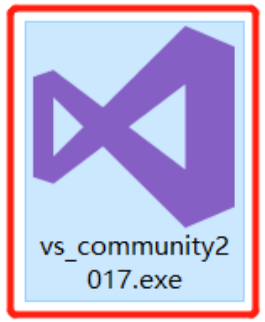
1.7 PX4 SITL软件在环仿真（不需要Pixhawk硬件）

- 该功能仅限RflySim高级版（对应步骤1.5中设置）
- 双击桌面上的“SITLRun”快捷方式，输入数字“1”，启动一个飞机的软件在环仿真系统
- 同上一页PPT，在QGC中控制飞机起飞，如果能够正常自动起飞，说明平台配置正确。
- 原理：PX4 SITL是在Win10WSL的Ubuntu环境中模拟了Pixhawk的实时操作系统，从而运行完整的PX4控制器，并与CopterSim通过网络相连实现传感器/控制指令的交互，形成控制仿真闭环系统，和插入Pixhawk硬件进行HITL效果相同
- 注：PX4 SITL仿真下也可以通过QGC来配置参与与获取log文件（存储在安装目录下的：Firmware\build\px4_sitl_default\instance_1下）



1.软硬件配置

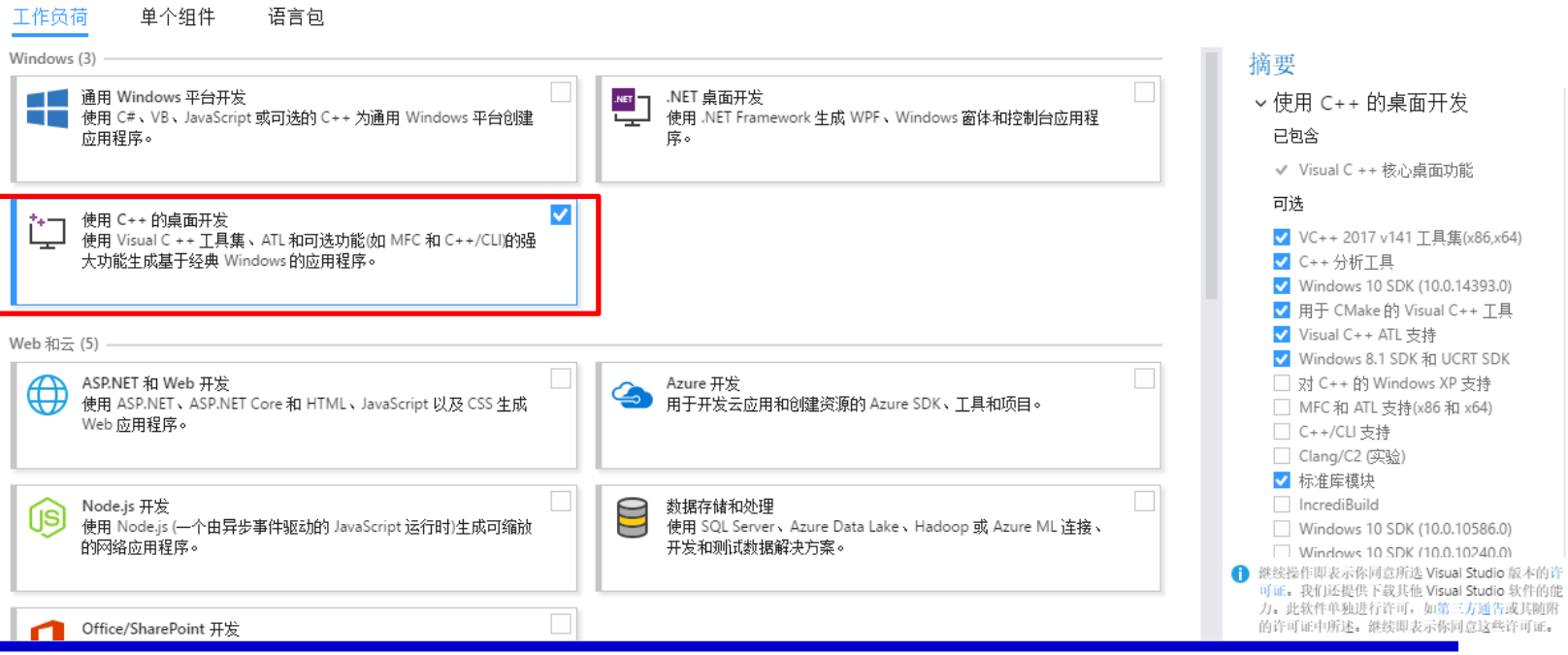
SimulinkControlAPI › VS2017Installer



1.8 安装Visual Studio 2017 (也可以用其他版本，MATLAB能识别即可)

- 后续课程很多地方都需要用到Visual Studio编译器，例如MATLAB S-Function Builder模块的使用、Simulink自动生成C/C++模型代码等
- 这里推荐安装Visual Studio 2017，在线安装步骤（需联网）如下：
- 双击“RflySimAPIs\SimulinkControlAPI\VS2017Installer\vs_community2017.exe”

- 本课程内容只需勾选右图的“C++的桌面开发”即可。
- 注意：如果后续要用于UE4 C++开发，还可以在右侧“摘要”里面勾选一个最新的Window 10 SDK；再点击“单个组件”标签页，勾选.NET 4.7.2（最新版即可）和对应pack包。再点安装。



1.软硬件配置

1.9 为MATLAB配置C++编译器

- 在MATLAB的命令行窗口中输入指令“mex -setup”
- 一般来说会自动识别并安装上VS 2017编译器，如右图所示显示“MEX配置使用‘Microsoft Visual C++ 2017’以进行编译”说明安装正确
- 本页面还可以切换选择VS 2015等其他编译器



大纲

1.软硬件配置

2.MAVLink通信解析

3.PX4官方控制器通信

4.代码生成控制器通信

5.小结

本小节的例程源码路径：
RflySimAPIs\SimulinkControlAPI\MavlinkDemo

2.MAVLink通信解析

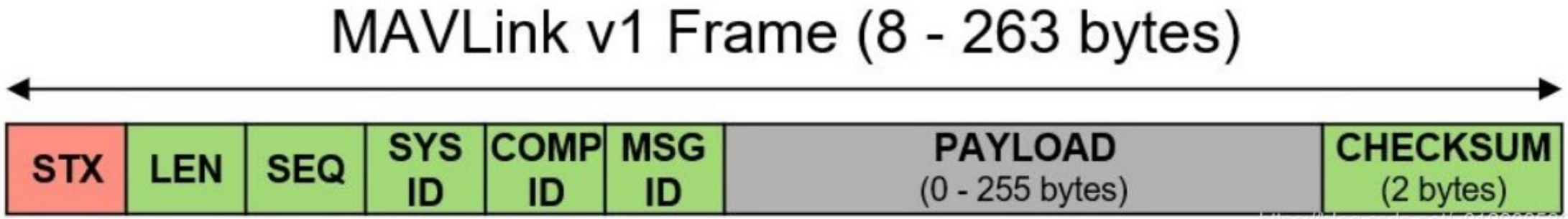
- 2.1 MAVLink（Micro Air Vehicle Link）是一种用于小型无人载具的通信协议，于2009年首次发布。该协议广泛应用于地面站（Ground Control Station，GCS）与无人载具（Unmanned vehicles）之间的通信，同时也应用在载具内部子系统的内部通信中，协议以消息库的形式定义了参数传输的规则。MAVLink协议支持无人固定翼飞行器、无人旋翼飞行器、无人车辆等多种载具。

- 官方使用文档网站：
  https://mavlink.io/en/messages/common.html
- MAVLink源码：
  https://github.com/mavlink/mavlink
- 基于MAVLink的QGroundControl地面站源码：
  https://github.com/mavlink/qgroundcontrol

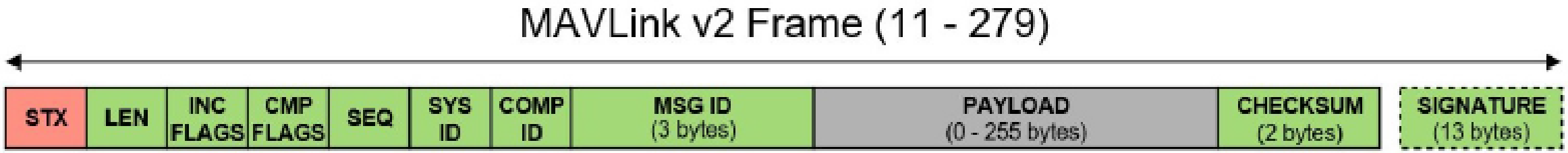
2.MAVLink通信解析

2.2 MAVLink的本质就是字节流的封装与解析协议

- MAVLink 1的数据包格式如下：



- MAVLink 2的数据包如下：



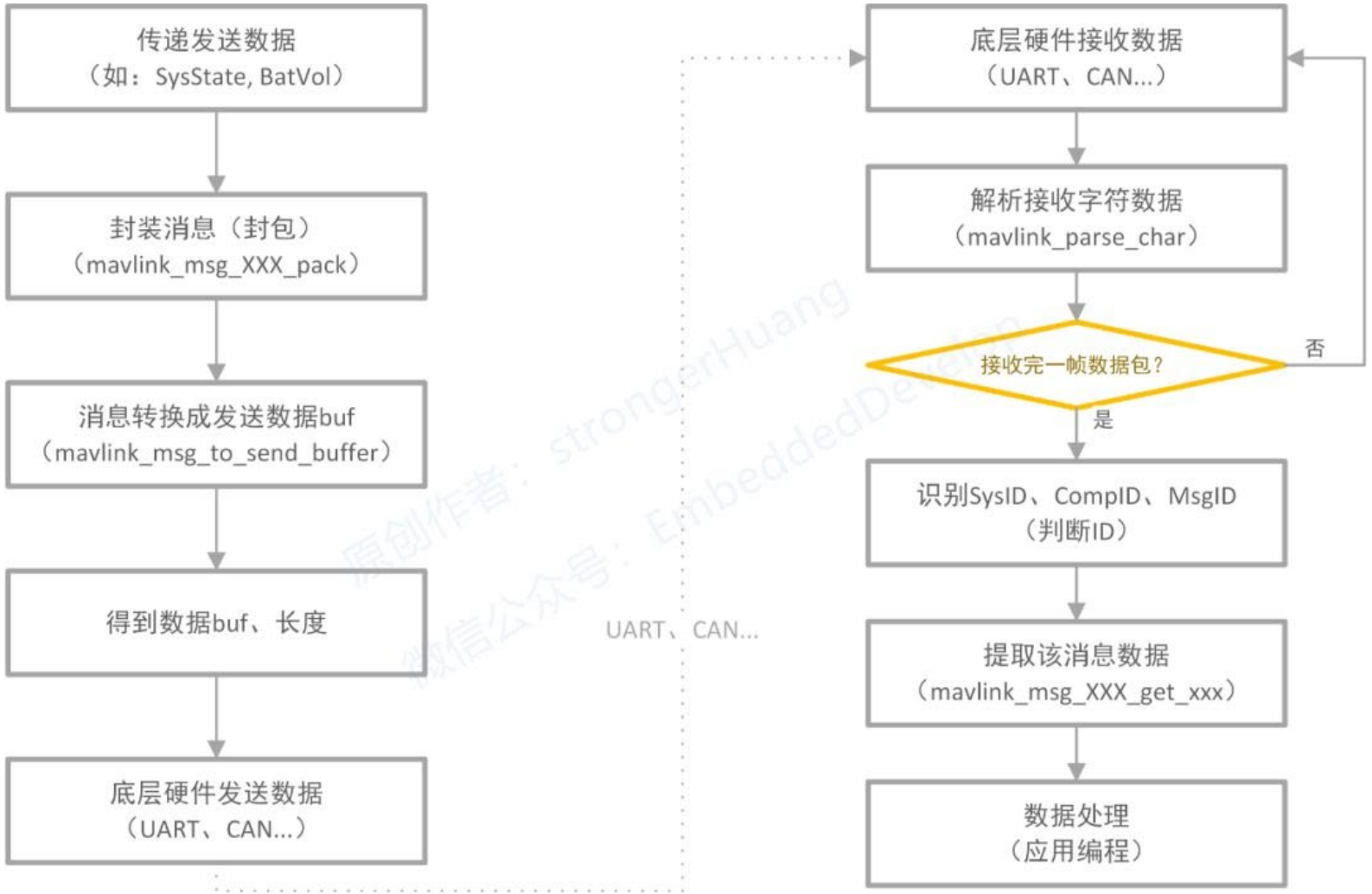
2.MAVLink通信解析

区域	名称	索引	长度	含义	取值
STX	起始标识	0	1	标识新消息的开始，用于消息识别、解析	254
LEN	负载长度	1	1	记录负载信息的长度	N
SEQ	序列码	2	1	消息发送序列码，用于通信可靠性检验	0-255
SYS	系统ID	3	1	发送该消息系统的系统ID	0-255
COMP	组件ID	4	1	发送该消息系统组件的组件ID	0-255
MSG	消息ID	5	1	标识该消息的种类	0-255
PAYLOAD	负载信息	6	1	消息内部负载信息	
CKA	校验位A	N+6	N	CRC校验位	自动
CKB	校验位B	N+7	1	CRC校验位	自动

2.MAVLink通信解析

2.3 解析原理

- 读取：所有字节流存入buffer，依次读取buffer中的字节数据，遇到STX标志位（MAVLink v1的标志位是0xFE，v2的标志位为0xFD）开始识别一条消息直到消息尾部，如果消息校验正确则将消息发送给处理程序
- 发送：按上一页PPT将消息转换成字节流





2.MAVLink通信解析

2.4 接收解析源码分析

给定一定长度的字节流buffer，长度为length，通过下列脚本解析，每解析出一个mavlink数据包就执行onMavLinkMessage函数

```
for(int i = 0 ; i < length ; ++i){
    msgReceived = mavlink_parse_char(MAVLINK_COMM_1, (uint8_t)buffer[i], &message, &status);
    if(msgReceived){
        emit onMavLinkMessage(message);
    }
}
```

其中：

```
void onMavLinkMessage(mavlink_message_t message);
```

是得到一个MAVLink消息包后的处理函数，需要根据这个消息的ID来识别当前包的用途（心跳包，GPS位置，姿态等），并提取出感兴趣的数据。



2.MAVLink通信解析

2.4 接收解析源码分析

解析函数实现如下，根据message.msgid跳到对应的_decode函数，解码出数据

```
void onMavLinkMessage(mavlink_message_t message){
   switch (message.msgid){
     case MAVLINK_MSG_ID_GLOBAL_POSITION_INT:{
       mavlink_global_position_int_t gp;
       mavlink_msg_global_position_int_decode(&message, &gp);
       outHilData.time_boot_ms = m_LastReceiveMavMsg;
       outHilData.GpsPos[0]=gp.lat;
       outHilData.GpsPos[1]=gp.lon;
       outHilData.GpsPos[2]=gp.alt;
       outHilData.relative_alt = gp.relative_alt;
       outHilData.GpsVel[0]=gp.vx;
       outHilData.GpsVel[1]=gp.vy;
       outHilData.GpsVel[2]=gp.vz;
       outHilData.hdg = gp.hdg;
       break;
     }
   }
}
```



2.MAVLink通信解析

2.5 发送源码解析 — 发送一条mavlink_hil_actuator_controls消息

```
void sendHILCtrlMessage(uint8_t modes, uint64_t flags, float ctrl[])
{
   mavlink_hil_actuator_controls_t hilctrl;
   hilctrl.mode = modes;
   hilctrl.flags = flags;
   for(int i=0;i<16;i++){
      hilctrl.controls[i]=ctrl[i];
   }
   mavlink_message_t mess;
   mavlink_msg_hil_actuator_controls_encode(SystemID, TargetCompID, &mess, &hilctrl);
   char buffer[500];
   memset(buffer,0,500);
   unsigned int length = mavlink_msg_to_send_buffer((uint8_t*)buffer, & mess);
   udp.writeDatagram(buffer,length);//通过UDP或者串口将buffer发送出去即可
}
```

2.MAVLink通信解析

2.6 MAVLink消息包的ID列表

- https://mavlink.io/en/messages/common.html

HEARTBEAT (#0) 心跳包，ID=0号消息

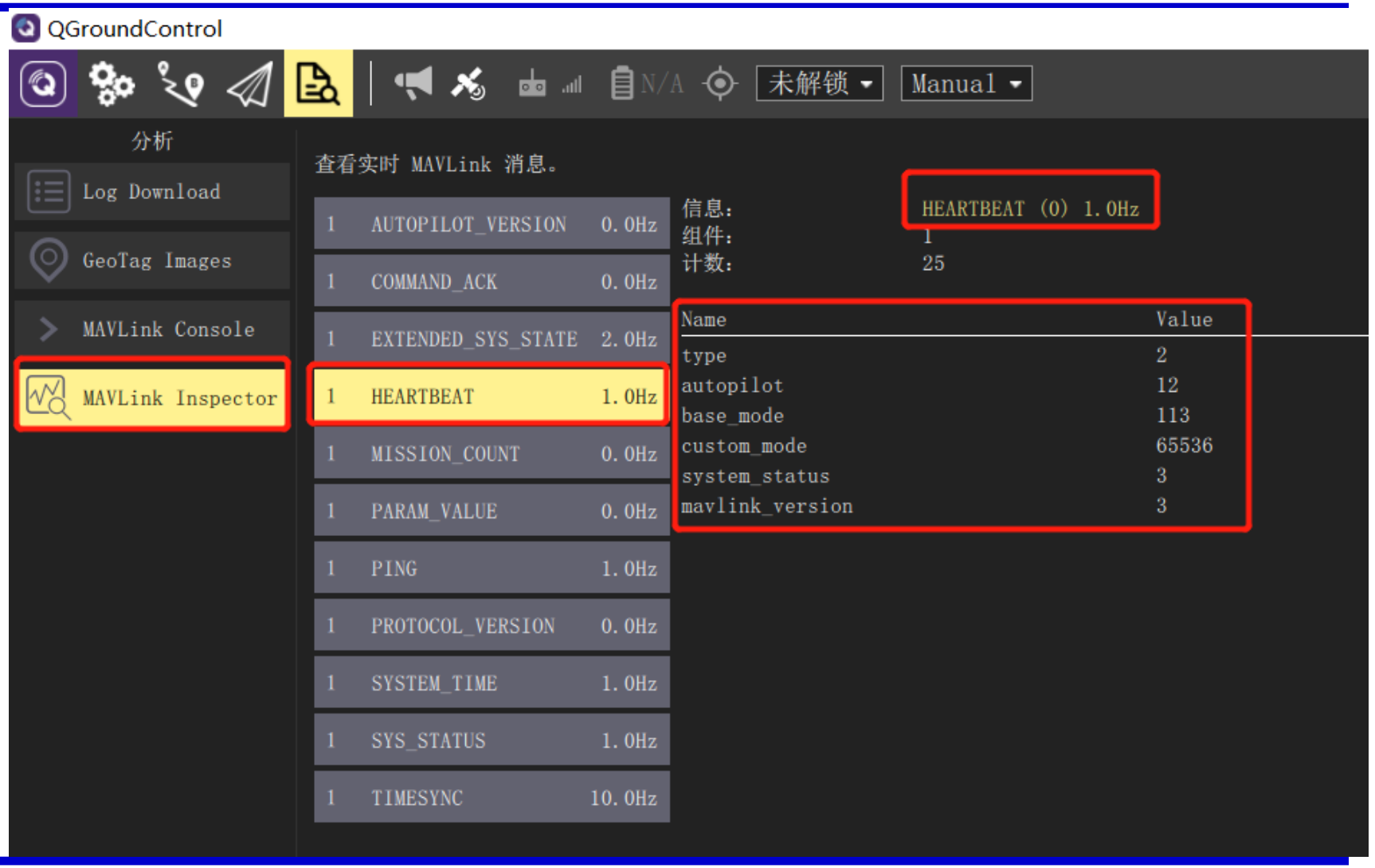
[Message] The heartbeat message shows that a system or component is present and responding. The type and autopilot fields (along with the message component id), allow the receiving system to treat further messages from this system appropriately (e.g. by laying out the user interface based on the autopilot). This microservice is documented at https://mavlink.io/en/services/heartbeat.html

Field Name	Type	Values	Description
type	uint8_t	MAV_TYPE	Vehicle or component type. For a flight controller component the vehicle type (quadrotor, helicopter, etc.). For other components the component type (e.g. camera, gimbal, etc.). This should be used in preference to component id for identifying the component type.
autopilot	uint8_t	MAV_AUTOPILOT	Autopilot type / class. Use MAV_AUTOPILOT_INVALID for components that are not flight controllers.
base_mode	uint8_t	MAV_MODE_FLAG	System mode bitmap.
custom_mode	uint32_t		A bitfield for use for autopilot-specific flags
system_status	uint8_t	MAV_STATE	System status flag.
mavlink_version	uint8_t_mavlink_version		MAVLink version, not writable by user, gets added by protocol because of magic data type: uint8_t_mavlink_version

2.MAVLink通信解析

2.7 QGC地面站查看MAVLink消息

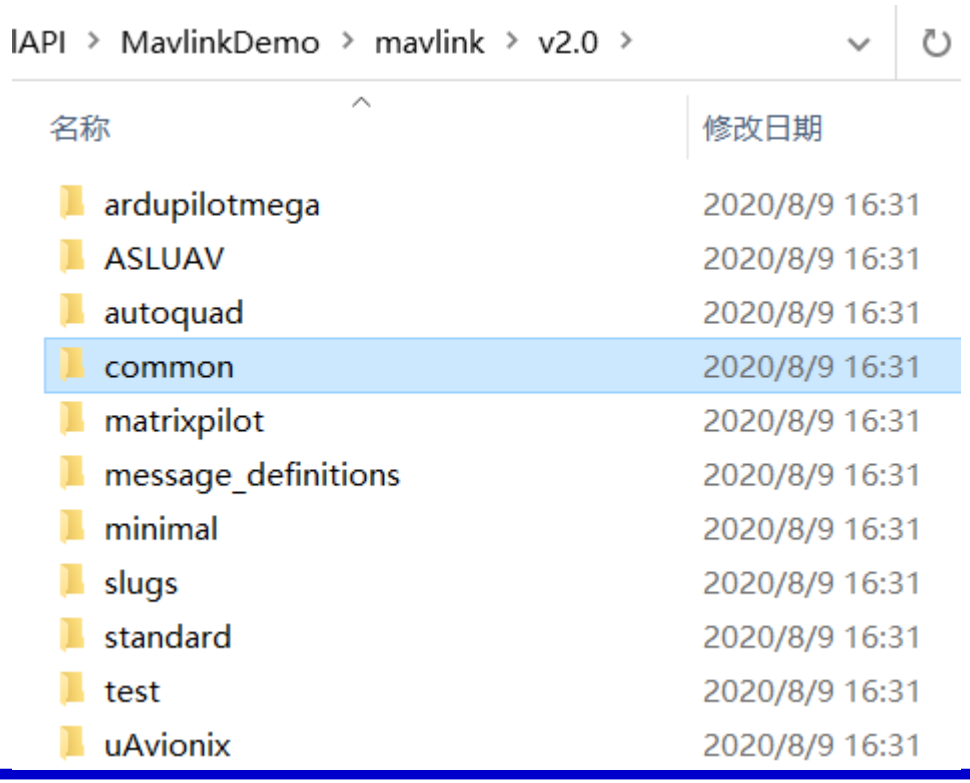
- 在QGC的MAVLink Inspector页面中可以浏览Pixhawk发送的所有MAVLink包，查看各个包的频率以及具体数值

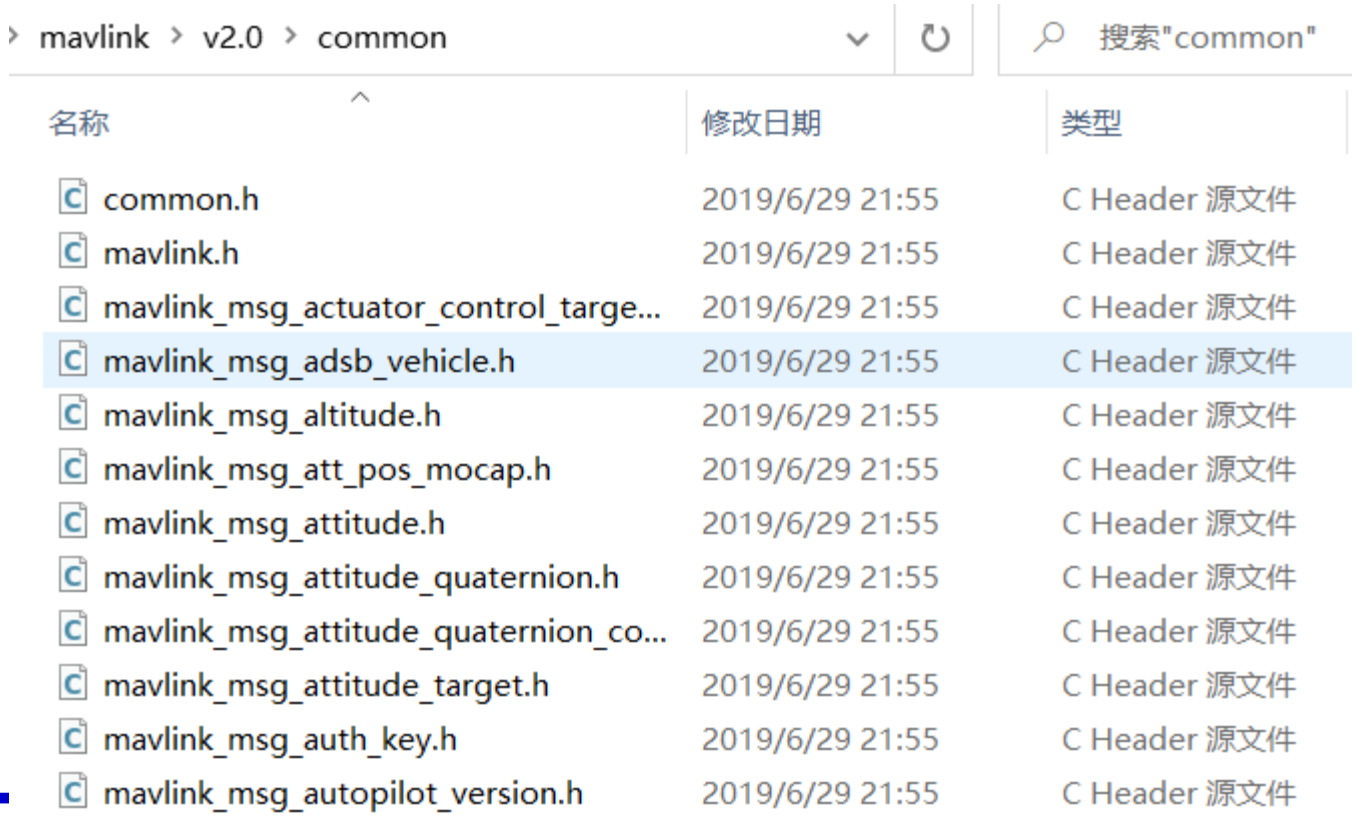


2.MAVLink通信解析

2.8 MAVLink 2的源码

- 打开文件夹“RflySimAPIs\SimulinkControlAPI\MavlinkDemo\mavlink\v2.0\common”可以看到MAVLink的C++源代码，里面包含了所有消息包的定义

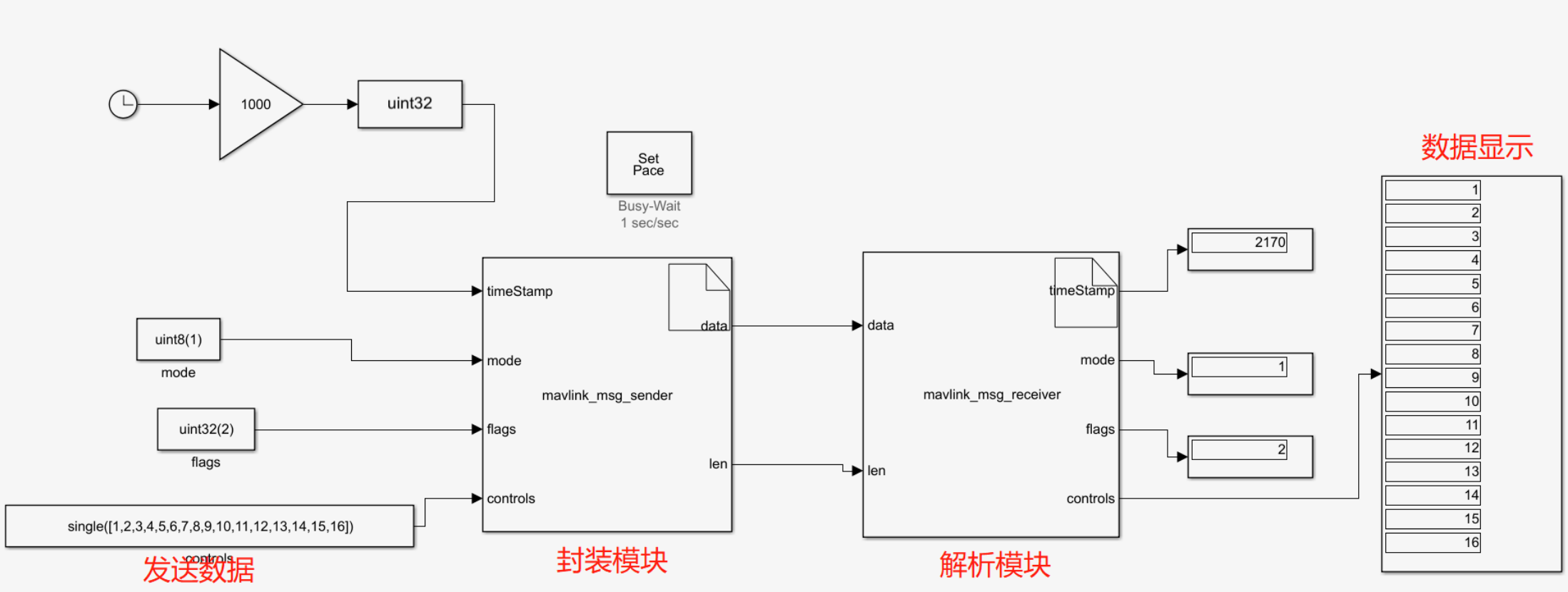




2.MAVLink通信解析

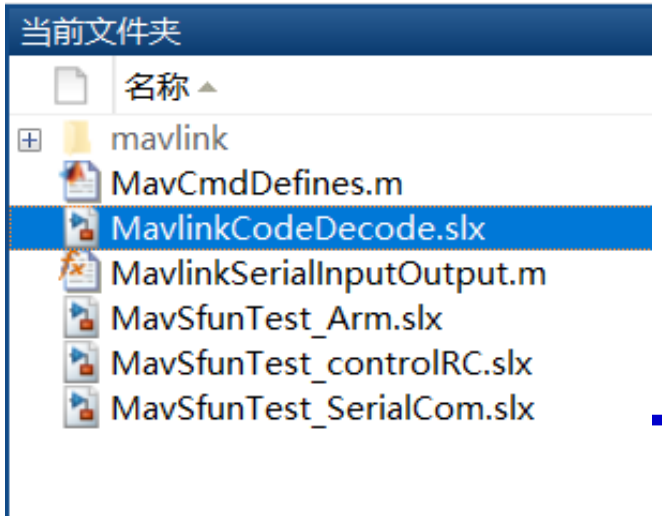
2.9 MAVLink协议的Simulink封装与解析实现

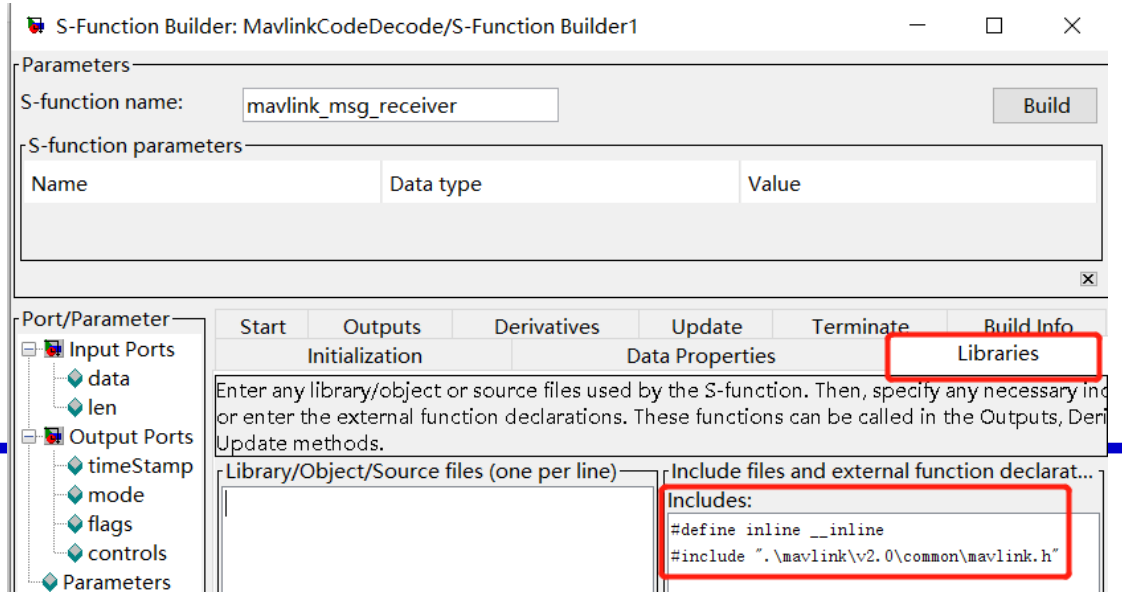
- 打开例程“RflySimAPIs\SimulinkControlAPI\MavlinkDemo\MavlinkCodeDecode.slx”

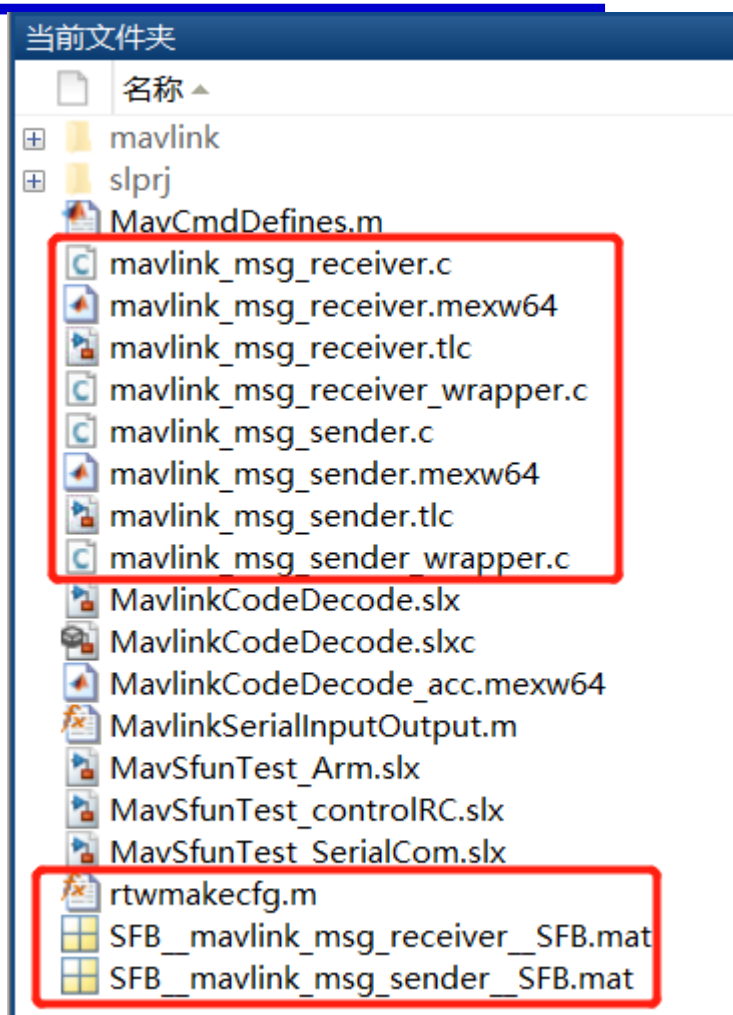


2.MAVLink通信解析

- 点击运行之后，可以看到我们在Simulink中将数据封装成了字节流data（uint8字节流）和len（字节流长度），然后又经过一个解析函数，将字节流解析成了发送数据。
- 本例子通过S-Function Builder实现，它在运行时会自动调用mavlink的头文件，并编译成右图所示的.c/.mexw64/.tlc等文件
- 本例程可以教会大家如何在Simulink中调用外部C/C++头文件来实现自己算法。



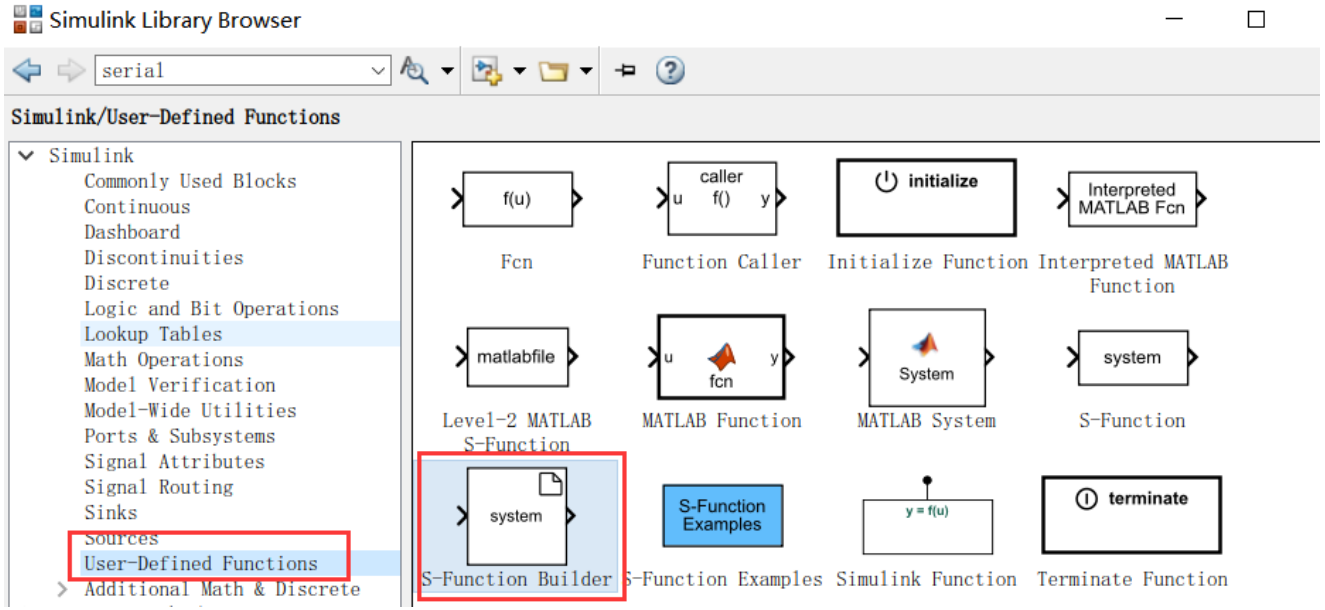


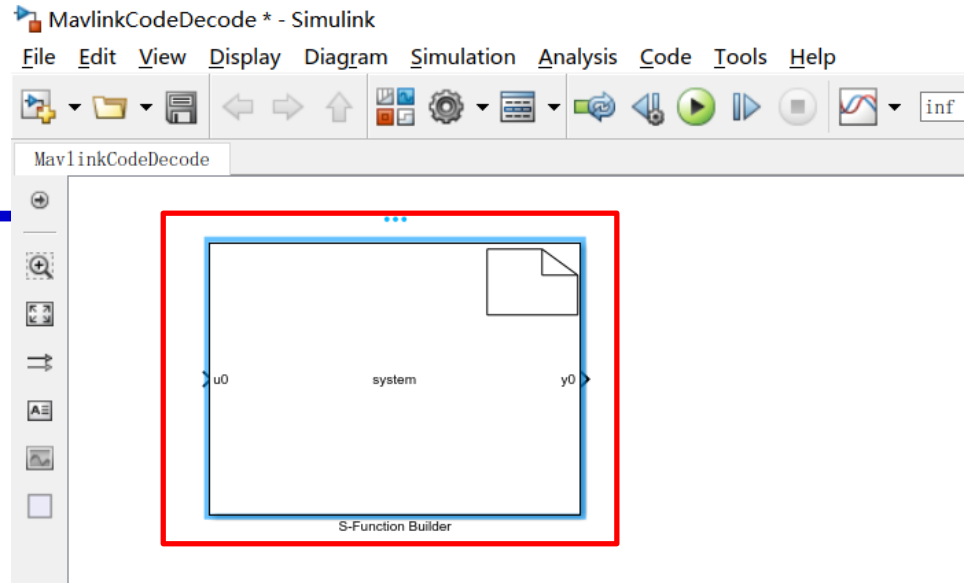


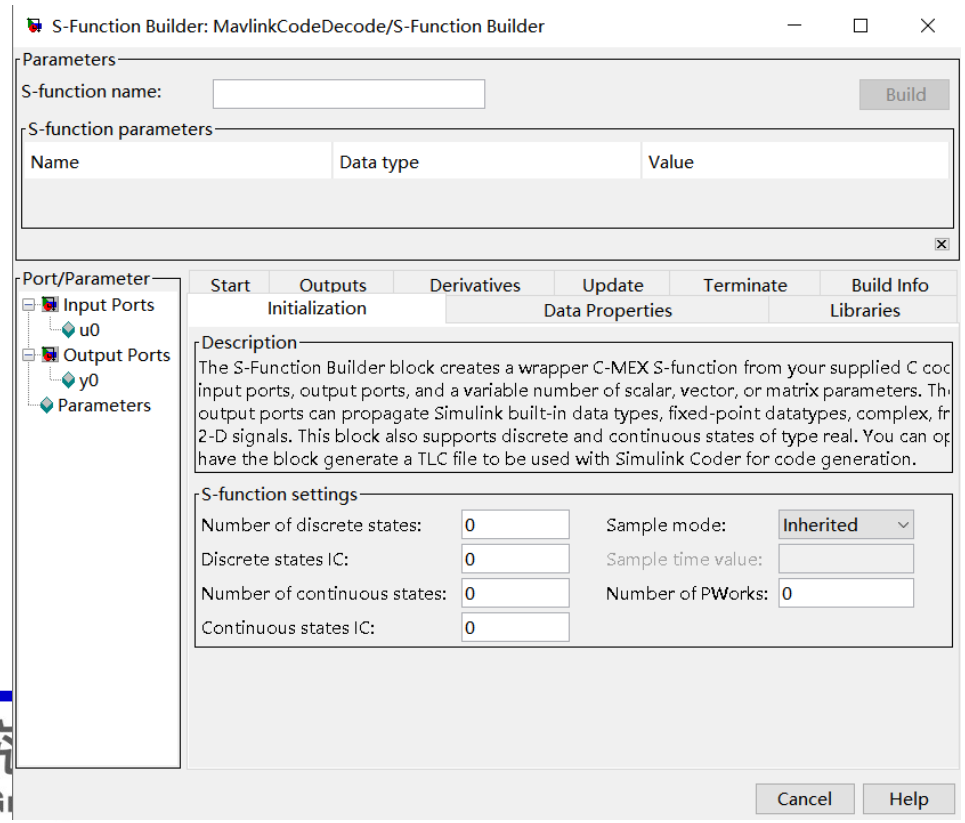
2.MAVLink通信解析

2.10 Simulink S-Function编写方法

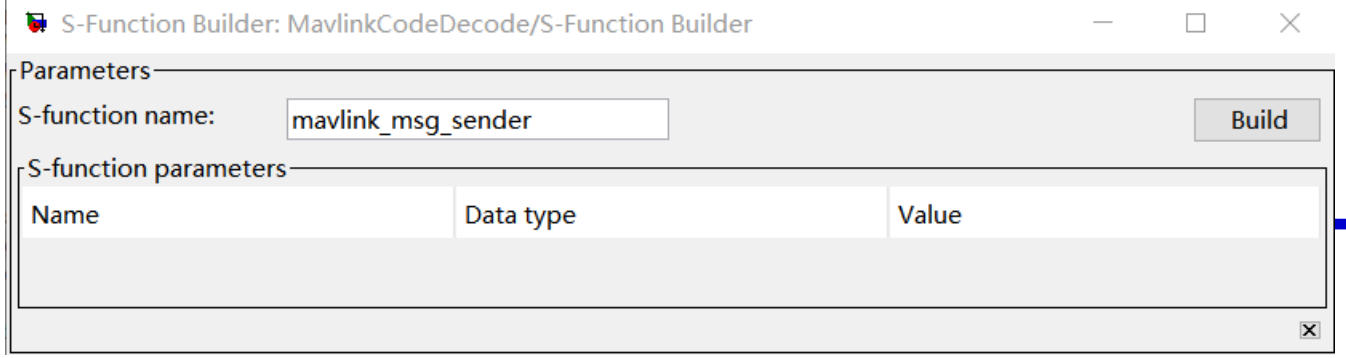
- 打开.slx文件，并从Simulink-User-Defined Functions中拖入一个S-Function Builder模块，双击它可以得到右图



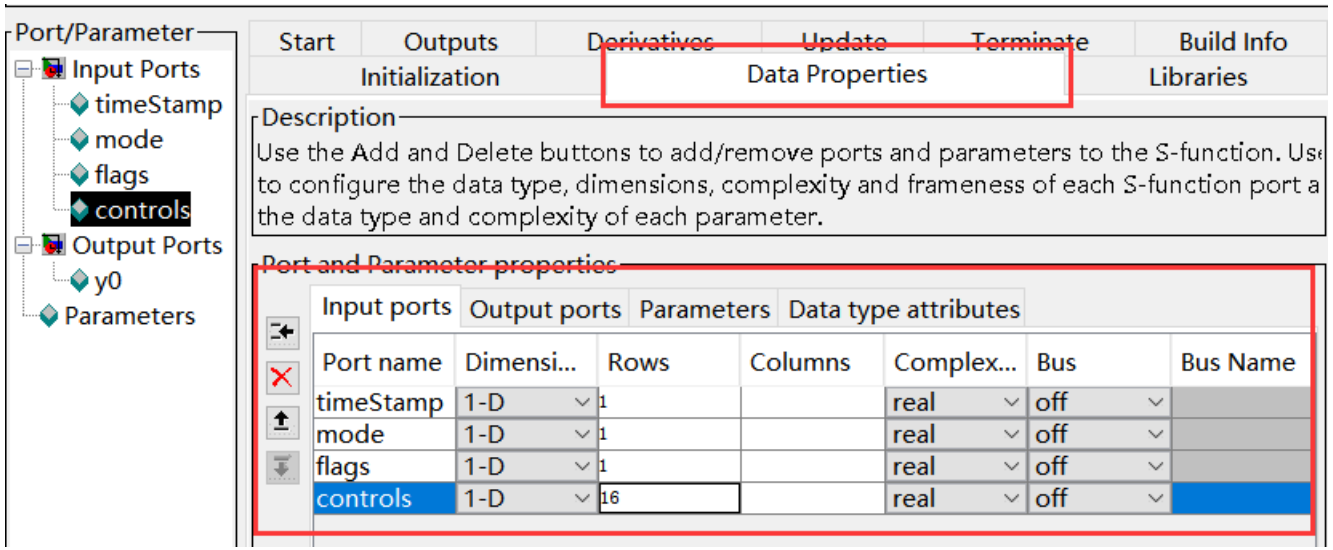


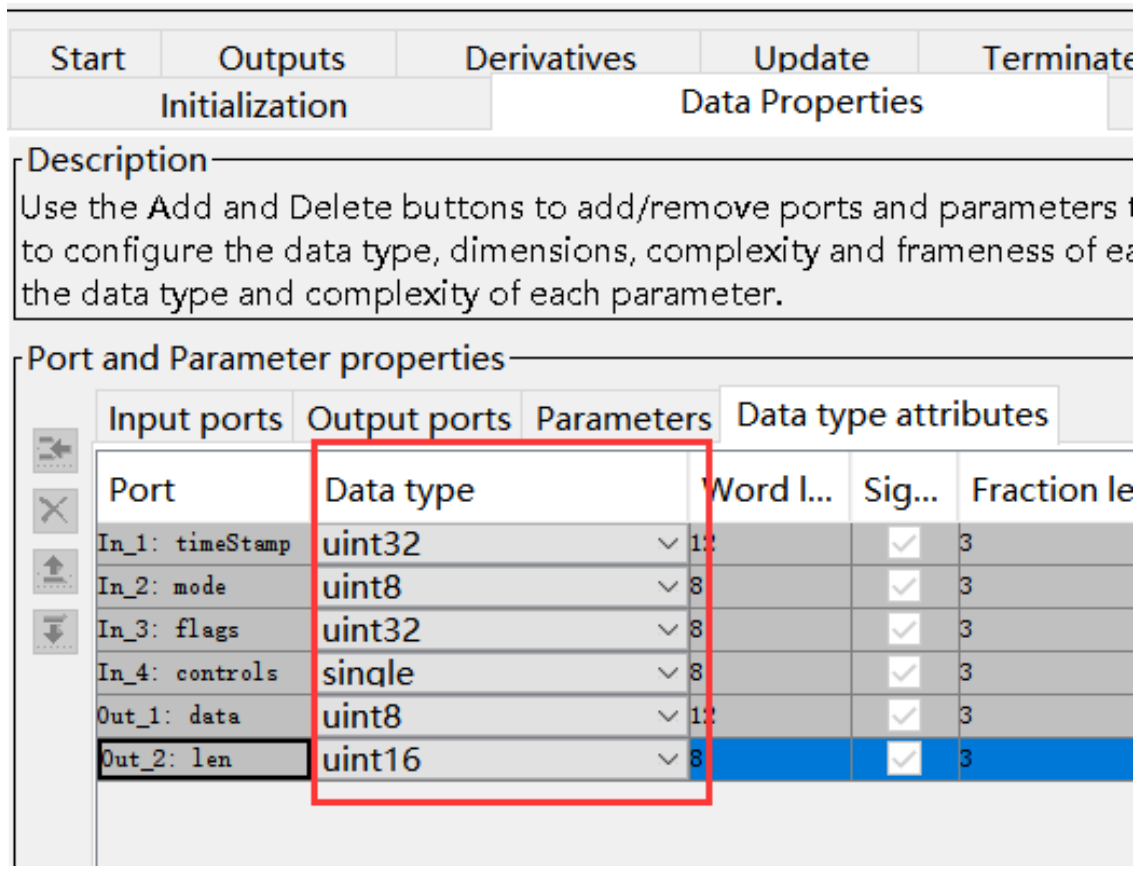


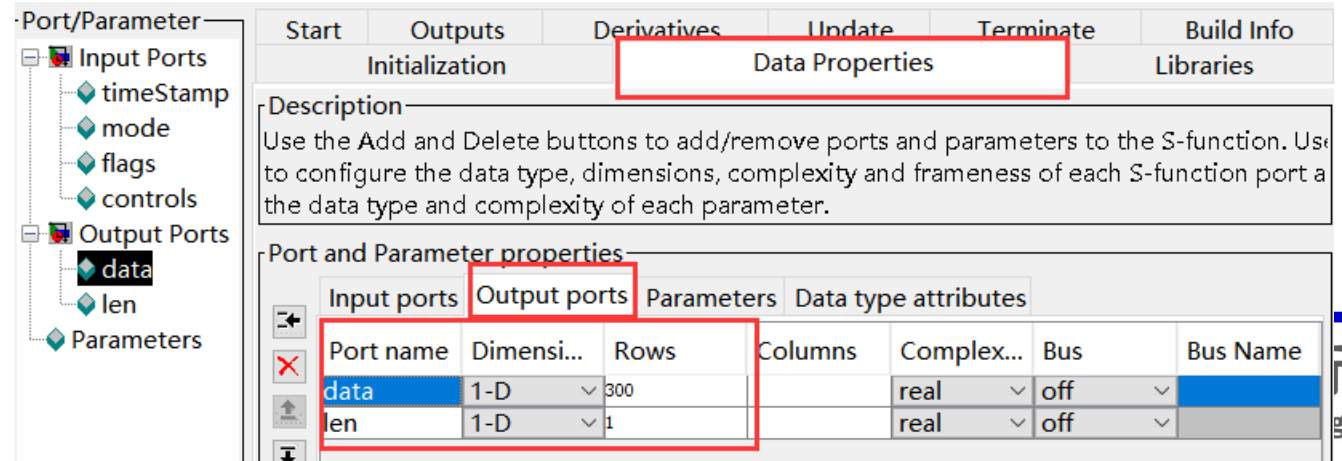
- 下面我们展示用这个模块生成前文提到的Mavlink消息：MAVLINK_MSG_ID_HIL_ACTUATOR_CONTROLS



- 对模块命名，并在DataProperties页面设置输入输出参数名称、维度、数据类型

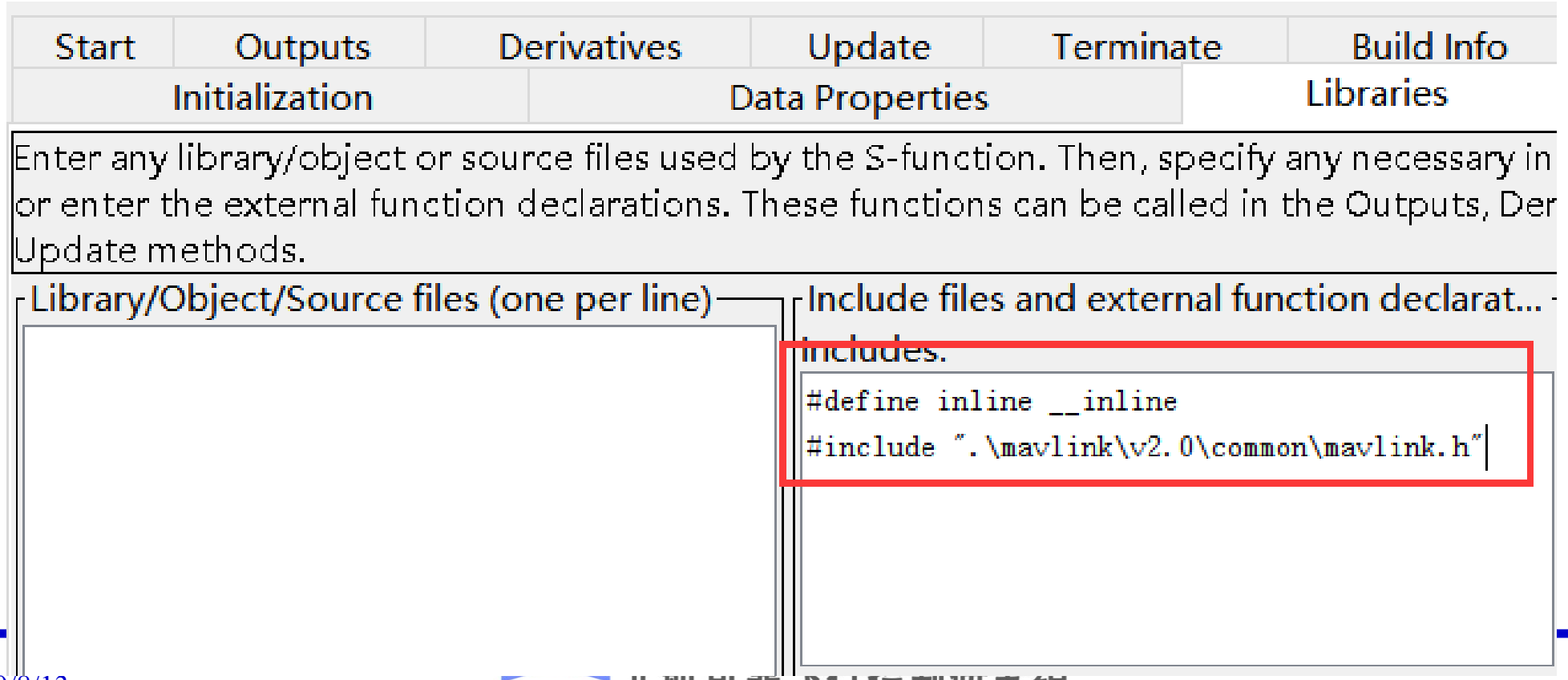






2.MAVLink通信解析

- 引入Mavlink头文件库在Libraries标签页的-includes框内加入下列代码
- #define inline __inline
- #include ".\mavlink\v2.0\common\mavlink.h"



2.MAVLink通信解析

- 在Outputs标签页中，添加获取输入数据，并打包成Mavlink消息的C/C++代码，放到输出口data和len中。其中data是uint8的矩阵，len是数据有效长度。
- 注意：Simulink S-function信号没有标量的概念，所有信号都是向量。因此len虽然维度是1，但是“len=***”这种幅值是错误的，要用“len[0]=***”。

Initialization | Data Properties | Libraries | Start | Outputs | Derivatives | Update | Terminate | Build Info

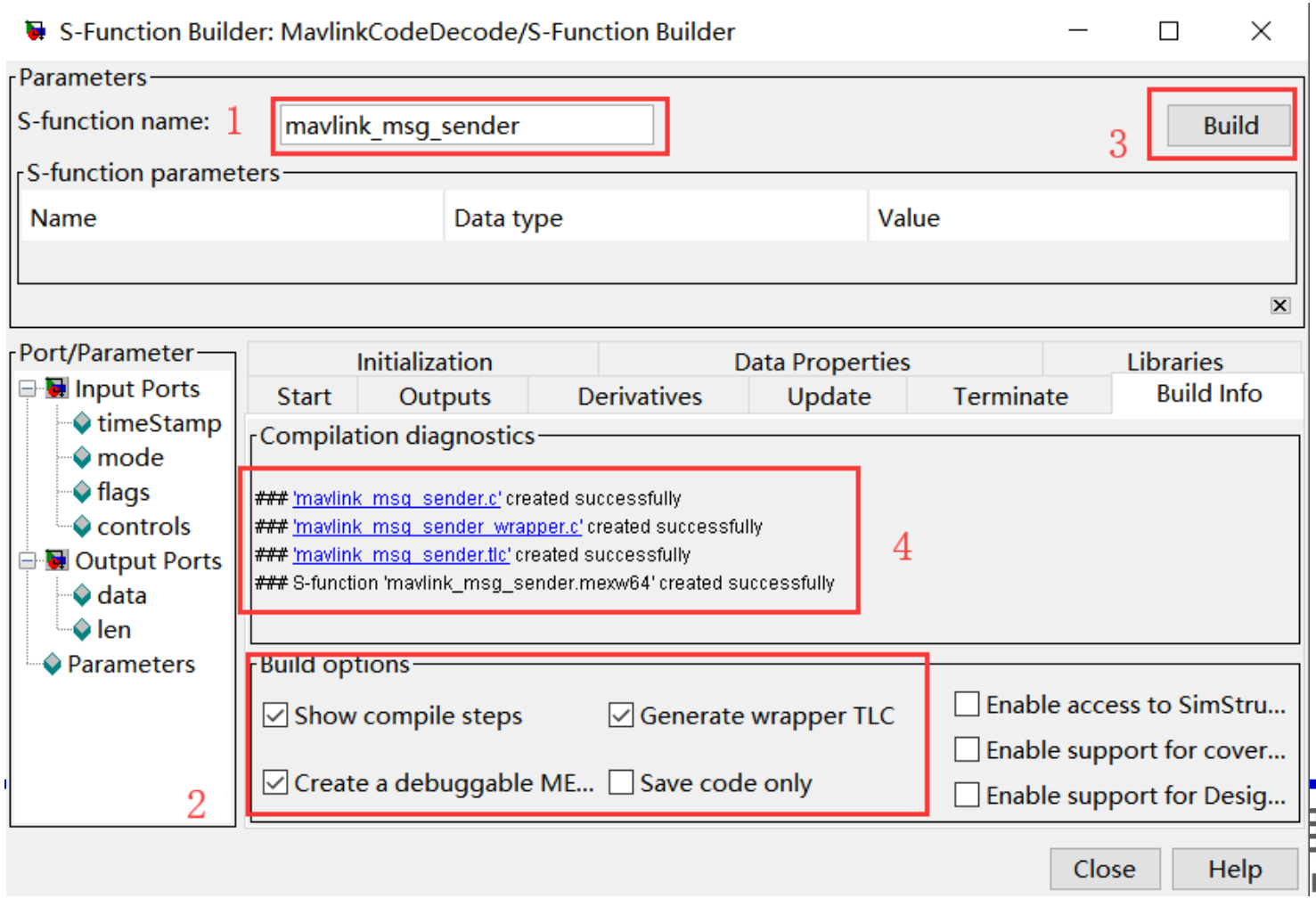
Code description

Enter your C-code or call your algorithm. If available, discrete and continuous states should be referenced as xD[0]...xD[n], xC[0]...xC[n] respectively. Input ports, output ports and parameters should be referenced using symbols specified in Data Properties. These references appear directly in the generated S-function.

```
uint8_T out_msg[MAVLINK_MAX_PACKET_LEN]={0};
mavlink_message_t out_msg_m1;
uint8_T sysid = 1;
uint8_T compid = 0;
mavlink_msg_hil_actuator_controls_pack(sysid,compid, &out_msg_m1,timeStamp[0],controls,mode[0],flags[0]);
uint16_t MavlinkMessageSizes = mavlink_msg_to_send_buffer(out_msg,&out_msg_m1);
len[0] = MavlinkMessageSizes;
for(int i=0;i<MavlinkMessageSizes;i++){  data[i]=out_msg[i]; }
```

2.MAVLink通信解析

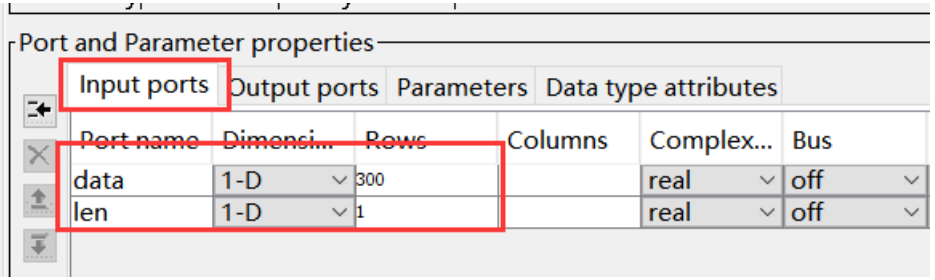
- 勾选生成TLC和MEX-file的选项，再点击编译按钮，就可以得到右图所示Simulink可调用的文件。

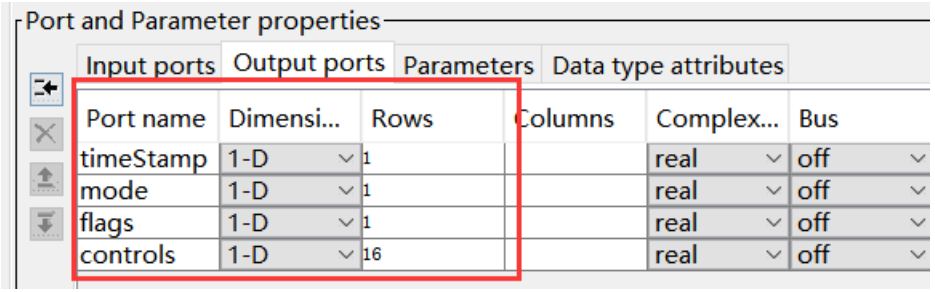


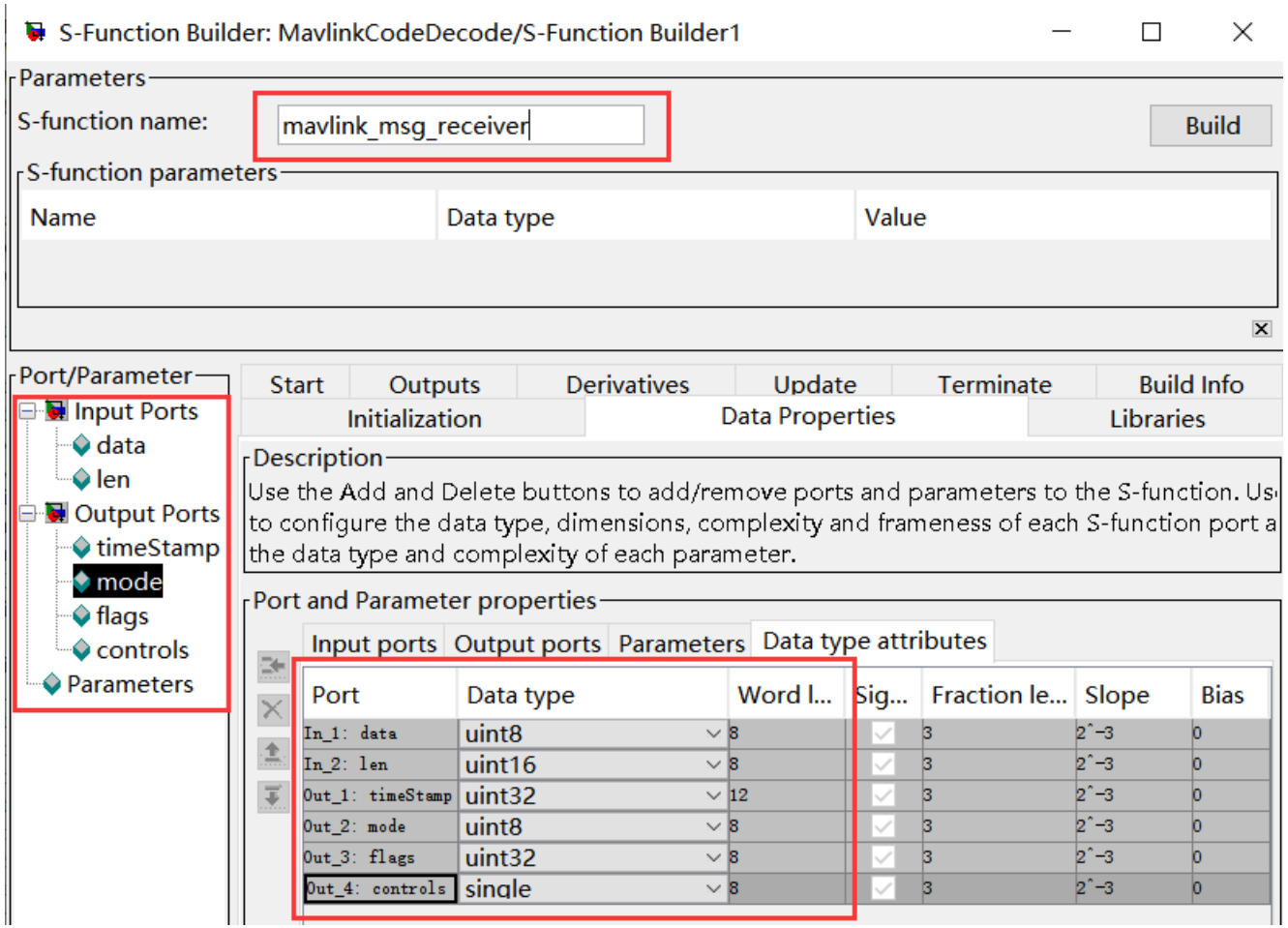


2.MAVLink通信解析

- 下面我们来构建一个Mavlink消息的解码模块
- 命名为“mavlink_msg_receiver”
- 输入输出口，和刚才的模块完全相反







2.MAVLink通信解析

- 如右图所示，在Outputs标签页中设置解码字节流并解析出Mavlink消息的代码。
- 同样的道理，在库文件页面，导入Mavlink库文件
- 设置好编译选项后，点击“build”按钮，查看是否可以正确生成tlc和mex文件。

Initialization | Data Properties | Libraries | Start | Outputs | Derivatives | Update | Terminate | Build Info

Code description

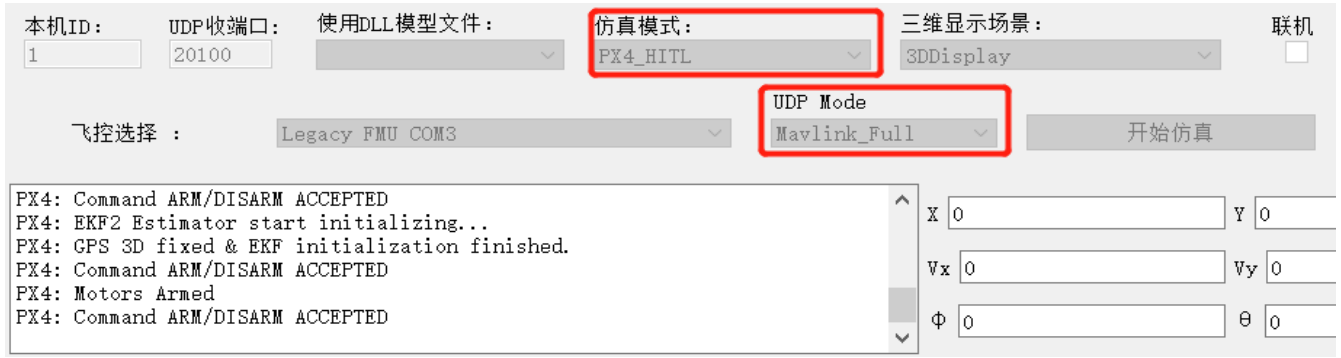
Enter your C-code or call your algorithm. If available, discrete and continuous states should be referenced as xD[0]...xD[n], xC[0]...xC[n] respectively. Input ports, output ports and parameters should be referenced using symbols specified in Data Properties. These references appear directly in the generated S-function.

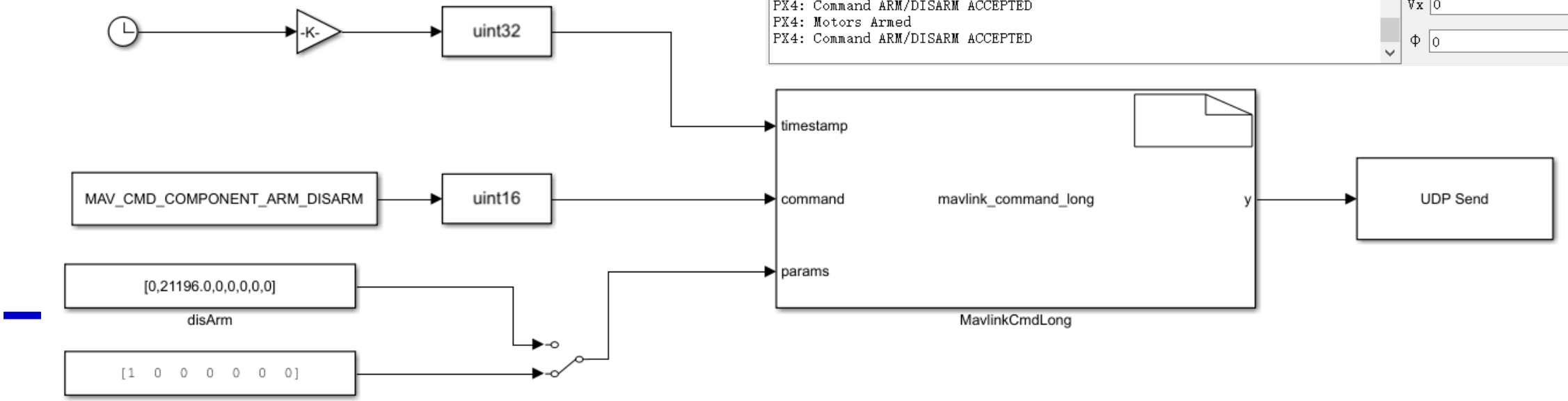
```
mavlink_message_t message;
mavlink_status_t status;
bool msgReceived = false;
for(int i=0;i<1en[0];i++){
    msgReceived = mavlink_parse_char(MAVLINK_COMM_1, (uint8_t)data[i], &message, &status);
    if(msgReceived){
        switch(message.msgid)
        {
            case MAVLINK_MSG_ID_HIL_ACTUATOR_CONTROLS:{
                mavlink_hil_actuator_controls_t hil_actuator_control;
                mavlink_msg_hil_actuator_controls_decode(&message, &hil_actuator_control);
                timeStamp[0] = hil_actuator_control.time_usec;
                mode[0] = hil_actuator_control.mode;
                flags[0] = hil_actuator_control.flags;
                for(int i=0;i<16;i++){
                    controls[i]=hil_actuator_control.controls[i];
                }
                break;
            }
            default:{
                break;
            }
        }
        msgReceived = false;
    }
}
```

2.MAVLink通信解析

2.11 通过Simulink/MAVLink向飞控发送解锁指令（仅限RflySim高级版）

- 插上Pixhawk，打开CopterSim，设置HITL仿真，UDP_Mode设置为Mavlink_FULL（仅限RflySim高级版），然后点击“开始仿真”按钮
- 打开例程“MavlinkDemo\MavSfunTest_Arm.slx”并运行，可以在CopterSim的消息框中看到“Command ARM/DISARM ACCECPTED”说明实验成功

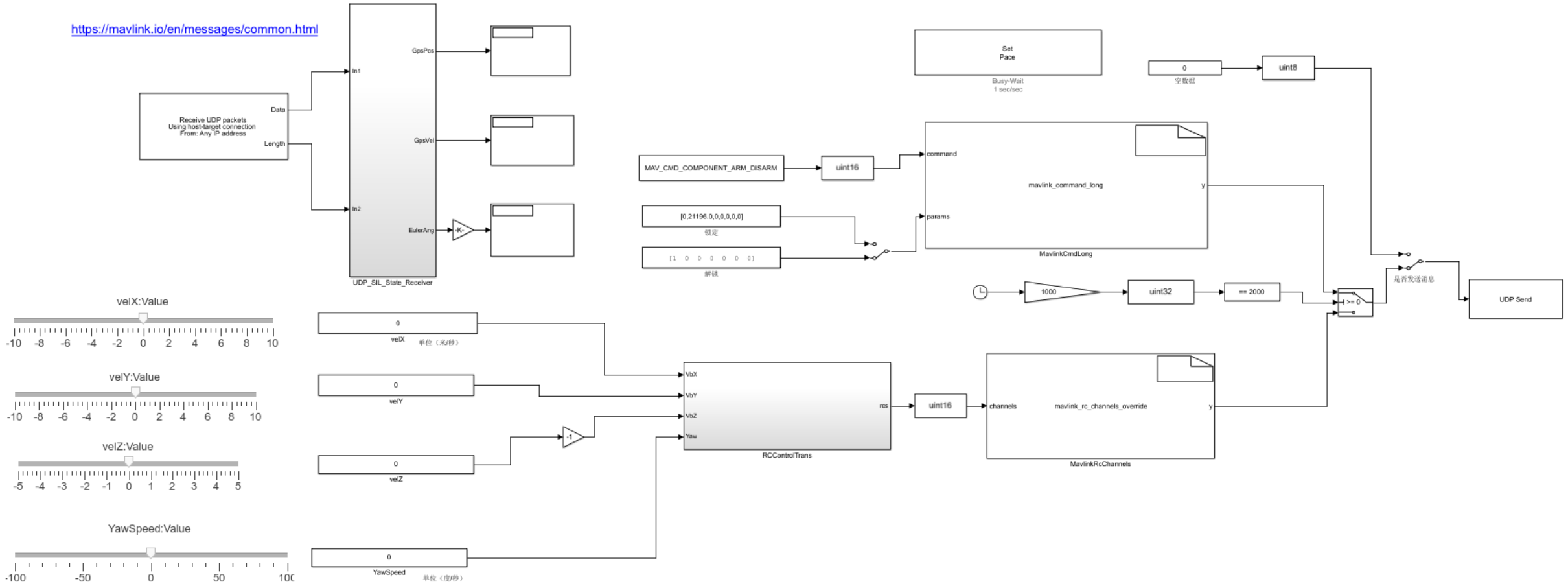




2.MAVLink通信解析

2.12 通过MAVLink模拟发送遥控器数据（仅限RflySim高级版）

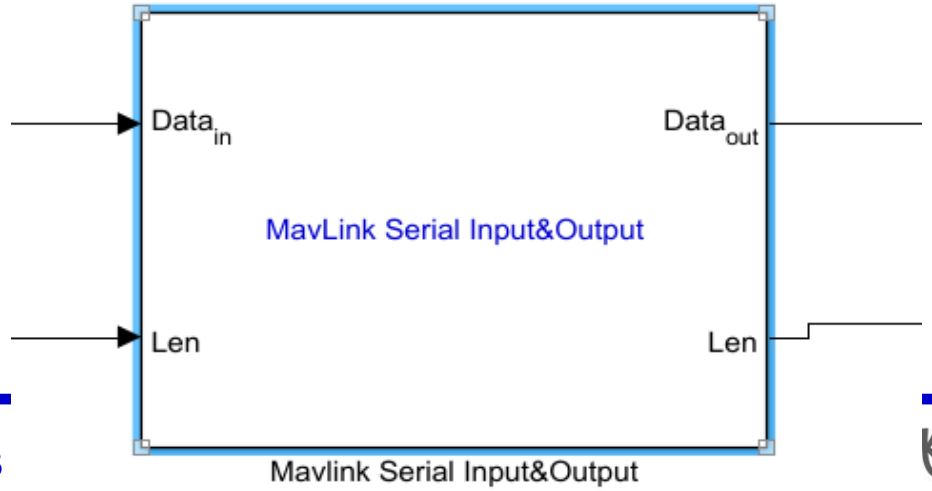
- 和上一步相同，打开CopterSim硬件在环，并运行“MavlinkDemo\MavSfunTest_Arm.slx”例程，可以控制Pixhawk的解锁，并发送遥控器数据，控制飞机起降、飞行等。

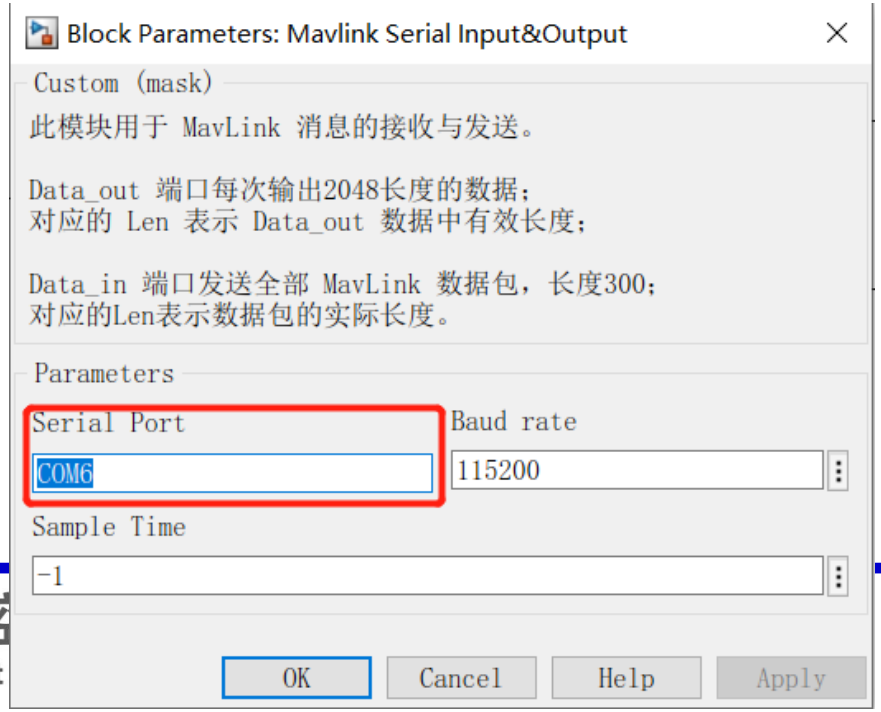


2.MAVLink通信解析

2.13 Simulink通过串口收发MAVLink消息

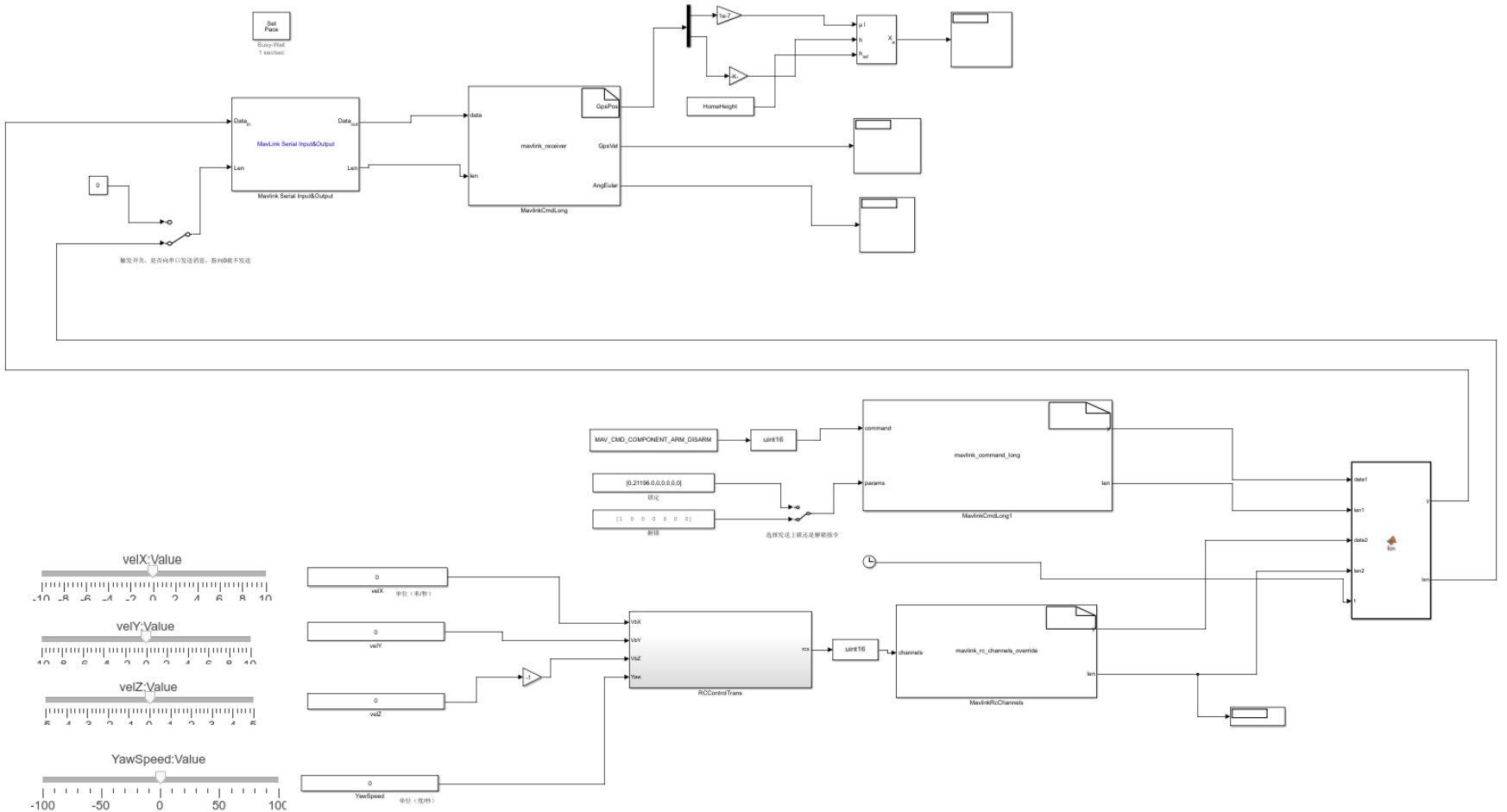
- 连接Pixhawk到电脑，用CopterSim开始硬件在环仿真（不需要设置UDP_Mode因此RflySim基础版也适用），用一个数传模块连接Pixhawk到电脑，记住数传模块的串口号（如果没有数传，可以不打开CopterSim，直接插上Pixhawk，这里输入Pixhawk的串口号也行）。
- 打开“MavlinkDemo\MavSfunTest_SerialCom.slx”，双击“Mavlink Serial Input&Output”，在其中输入数传串口号





2.MAVLink通信解析

- 在本例中可以通过串口获取Pixhawk的数据，并发送控制指令。本例子可以直接用于Pixhawk多旋翼的真机实时（通过数传）控制。



大纲

1.软硬件配置

2.MAVLink通信解析

3.PX4官方控制器通信

4.代码生成控制器通信

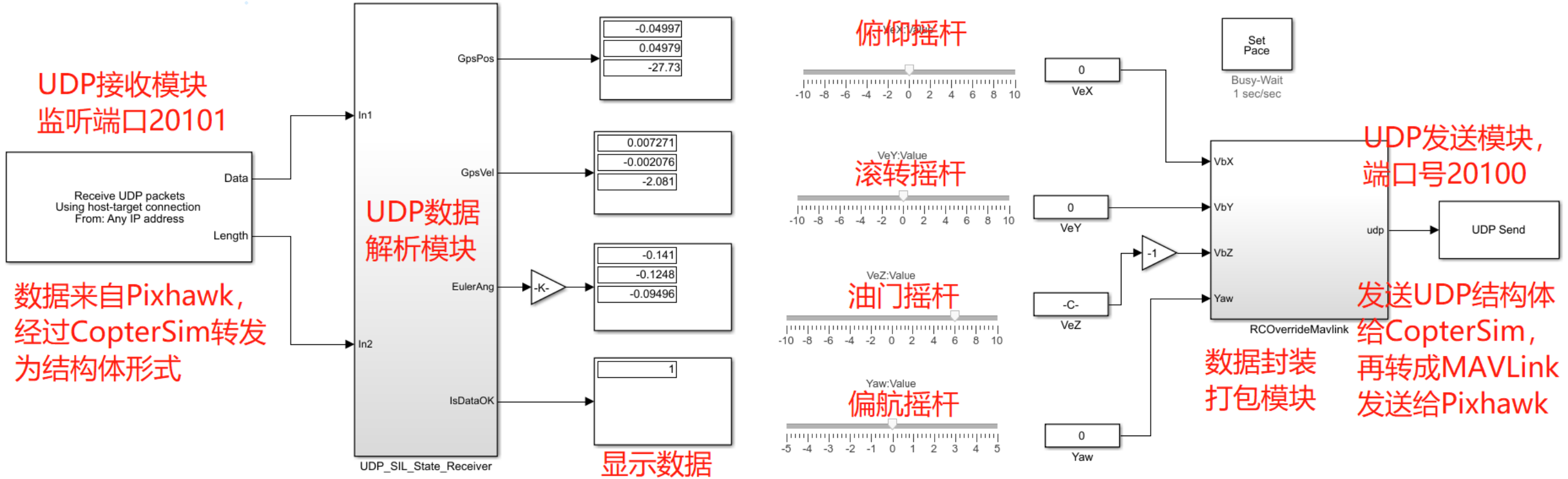
5.小结

本小节的例程源码路径：
RflySimAPIs\SimulinkControlAPI

3.PX4官方控制器通信

3.1 Simulink模拟发送遥控器原始数据控制无人机

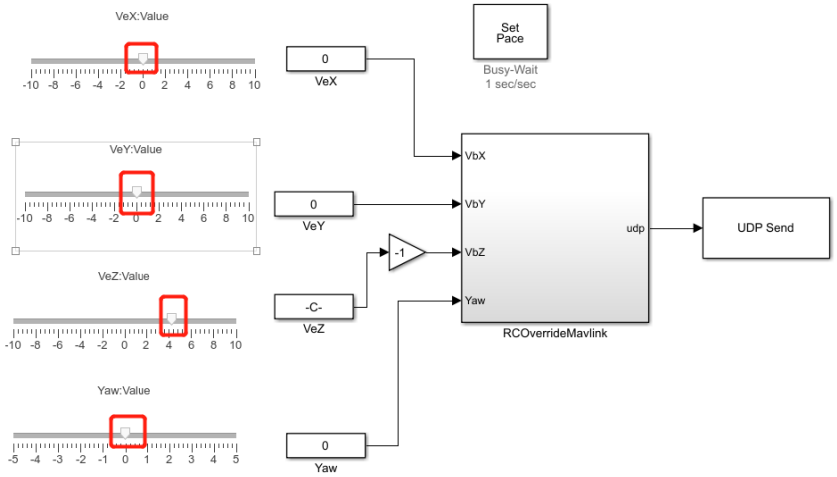
- 首先用CopterSim连接Pixhawk并开始硬件在环仿真同时打开三维软件（RflySim高级版可以直接运行桌面HITLRun一键快速打开硬件在环，或运行SITLRun一键打开软件在环仿真），然后通过MATLAB打开“RflySimAPIs\SimulinkControlAPI\RadioControlAPI.slx”文件，并运行。



3.PX4官方控制器通信

3.1 Simulink模拟发送遥控器原始数据控制无人机

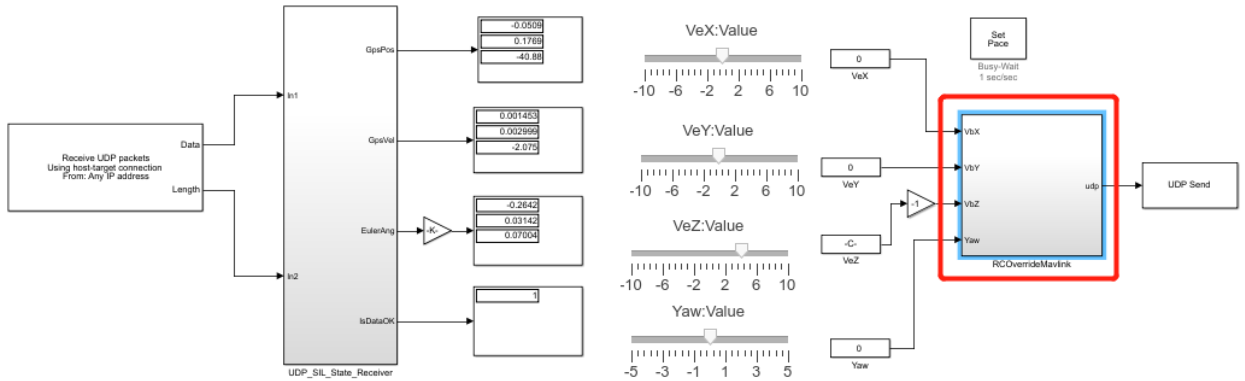
- 首先，向右拖动VeZ滑块（模拟向上推油门过中点），可以控制飞机Z方向速度，使飞机垂直起飞；
- 然后，拖动VeX（模拟前后推俯仰摇杆）和VeY（模拟左右拨滚转摇杆）滑块可以实现前后与左右运动，
- 同样，拖动Yaw滑块（模拟左右拨偏航摇杆）可以控制偏航速度，控制飞机偏转。

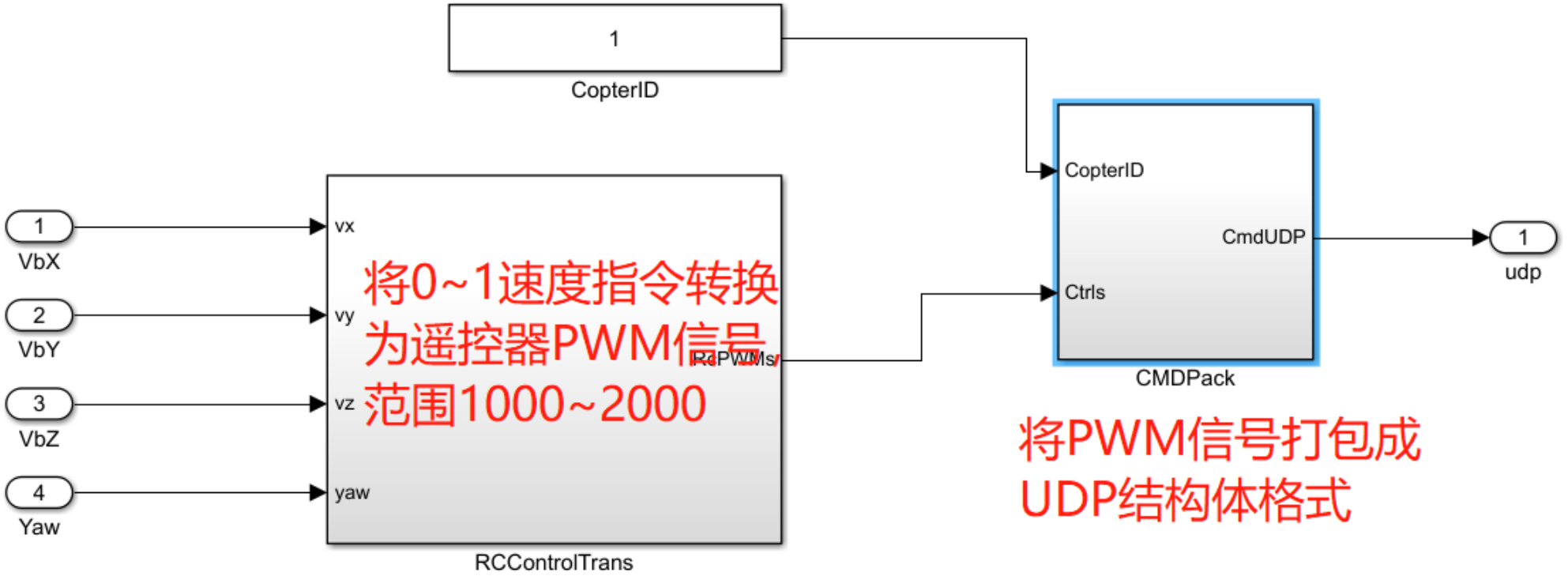




3.PX4官方控制器通信

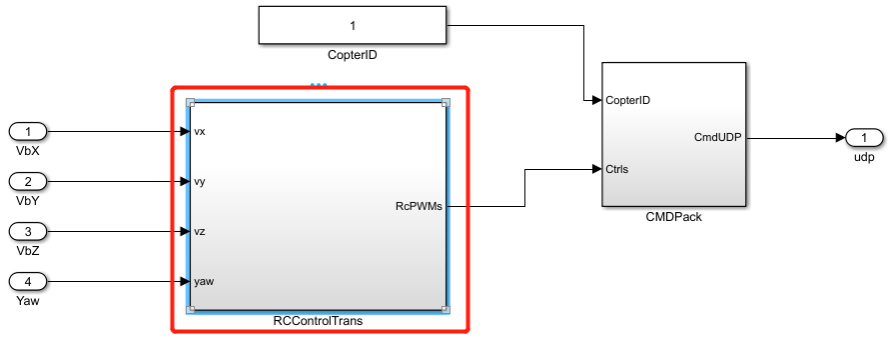
- 3.1 Simulink模拟发送遥控器原始数据
- 双击“RCOverrideMavlink”模块，可以看到下图所示的内部信息

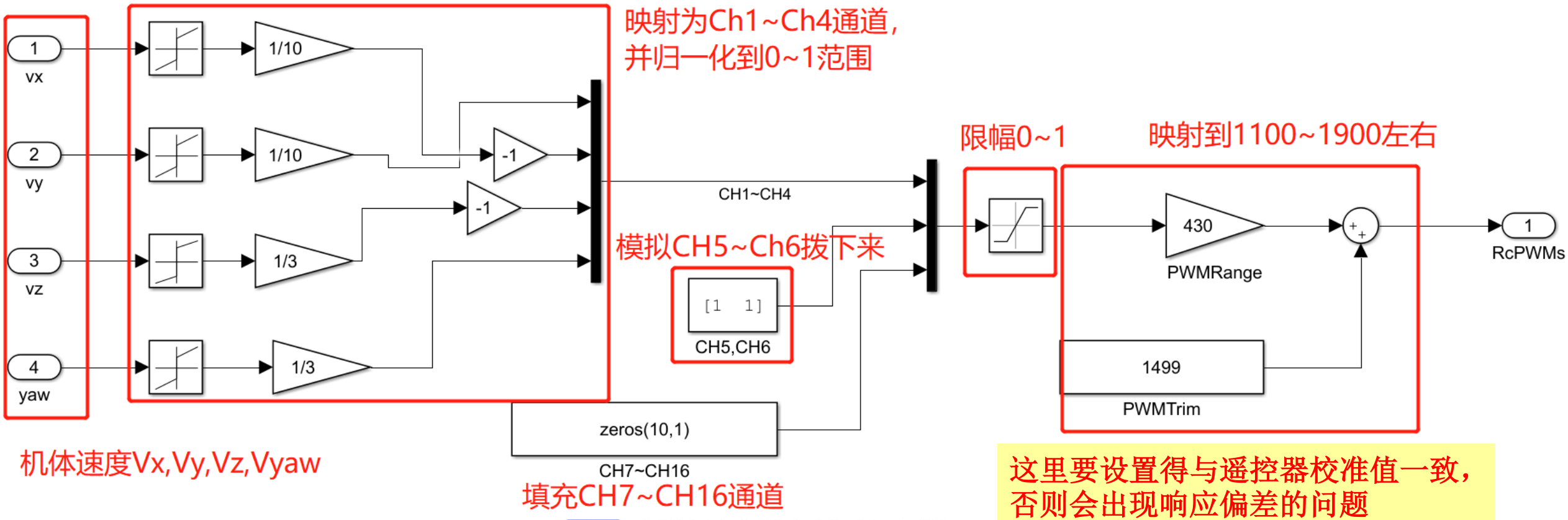




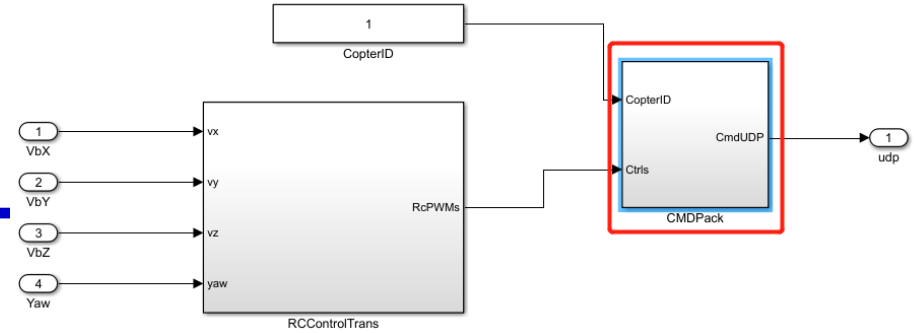
3.PX4官方控制器通信

- 3.1 Simulink模拟发送遥控器原始数据控制无人机
- 双击“RCControlTrans”模块，可以看到下图所示的内部信息





3.PX4官方控制器通信

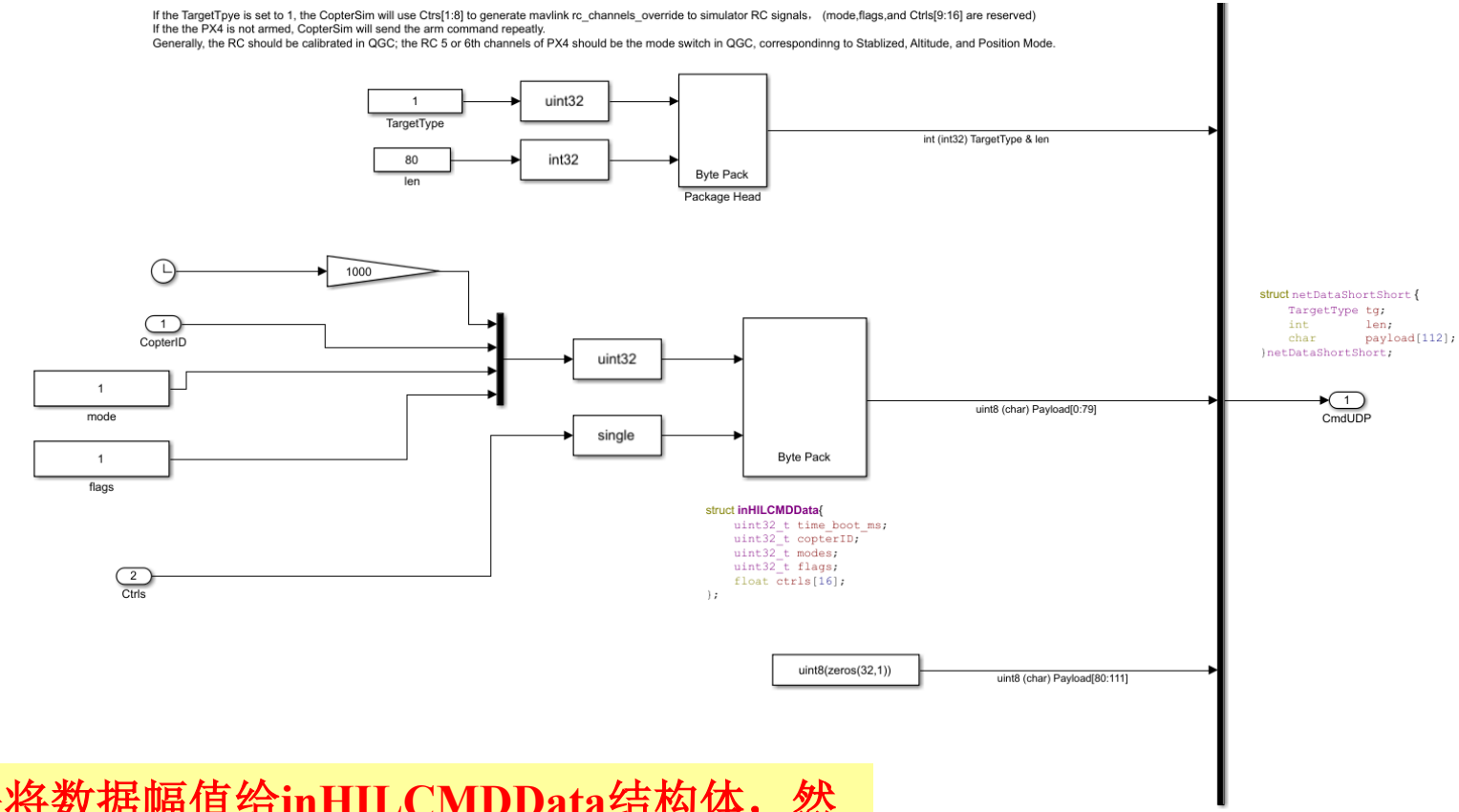


3.1 Simulink模拟发送遥控器原始数据控制无人机

- 双击进入“CMDPack”模块，可以看到下图所示的内部信息
- 两层封装（保证数据安全）

```
struct inHILCMDData{
    uint32_t time_boot_ms;
    uint32_t copterID;
    uint32_t modes;
    uint32_t flags;
    float ctrls[16];
};

struct netDataShortShort {
    TargetType tg;
    int        len;
    char       payload[112];
}netDataShortShort;
```



先将数据幅值给inHILCMDData结构体，然后将结构体数据存储到netDataShortShort结构体的payload数据段中，最后将数据发出去

3.PX4官方控制器通信

3.1 Simulink模拟发送遥控器原始数据控制无人机

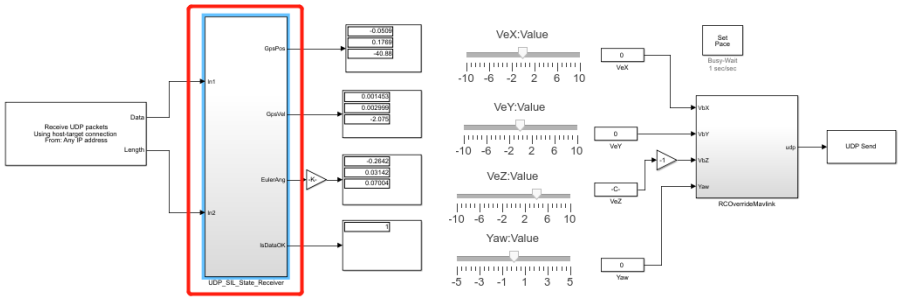
- CopterSim收到来自Simulink的UDP消息后，会生成MAVLink消息RC_CHAMMELS_OVERRIDE（遥控器器通道覆盖），并转发给Pixhawk实现遥控器信号的模块

RC_CHANNELS_OVERRIDE (#70)

The RAW values of the RC channels sent to the MAV to override info received from the RC radio. A value of UINT16_MAX means no change to that channel. A value of 0 means control of that channel should be released back to the RC radio. The standard PPM modulation is as follows: 1000 microseconds: 0%, 2000 microseconds: 100%. Individual receivers/transmitters might violate this specification.

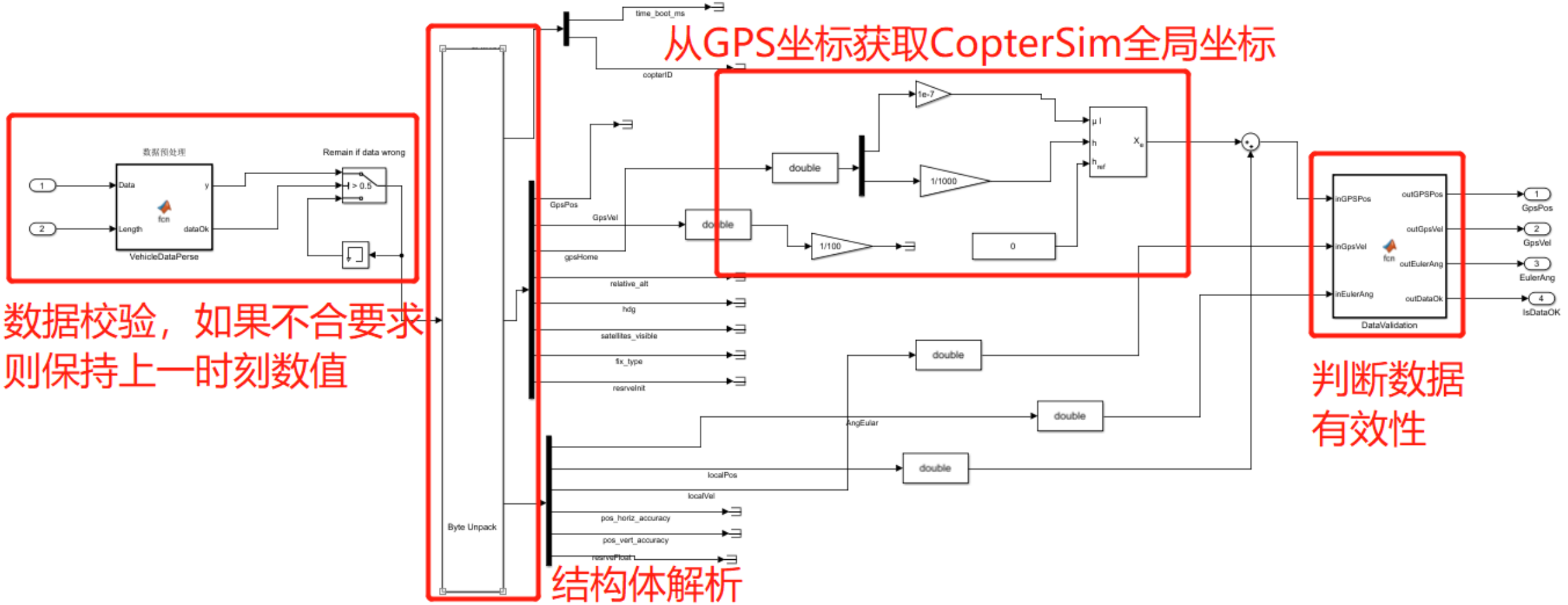
Field Name	Type	Units	Description
target_system	uint8_t		System ID
target_component	uint8_t		Component ID
chan1_raw	uint16_t	us	RC channel 1 value. A value of UINT16_MAX means to ignore this field.
chan2_raw	uint16_t	us	RC channel 2 value. A value of UINT16_MAX means to ignore this field.
chan3_raw	uint16_t	us	RC channel 3 value. A value of UINT16_MAX means to ignore this field.
chan4_raw	uint16_t	us	RC channel 4 value. A value of UINT16_MAX means to ignore this field.
chan5_raw	uint16_t	us	RC channel 5 value. A value of UINT16_MAX means to ignore this field.
chan6_raw	uint16_t	us	RC channel 6 value. A value of UINT16_MAX means to ignore this field.
chan7_raw	uint16_t	us	RC channel 7 value. A value of UINT16_MAX means to ignore this field.
chan8_raw	uint16_t	us	RC channel 8 value. A value of UINT16_MAX means to ignore this field.
chan9_raw **	uint16_t	us	RC channel 9 value. A value of 0 or UINT16_MAX means to ignore this field.
chan10_raw **	uint16_t	us	RC channel 10 value. A value of 0 or UINT16_MAX means to ignore this field.
chan11_raw **	uint16_t	us	RC channel 11 value. A value of 0 or UINT16_MAX means to ignore this field.

3.PX4官方控制器通信



3.1 Simulink模拟发送遥控器原始数据控制无人机

- 再回到最外层，点击进入“UDP_SIL_State_Receiver”模块



3.PX4官方控制器通信

3.1 Simulink模拟发送遥控器原始数据控制无人机

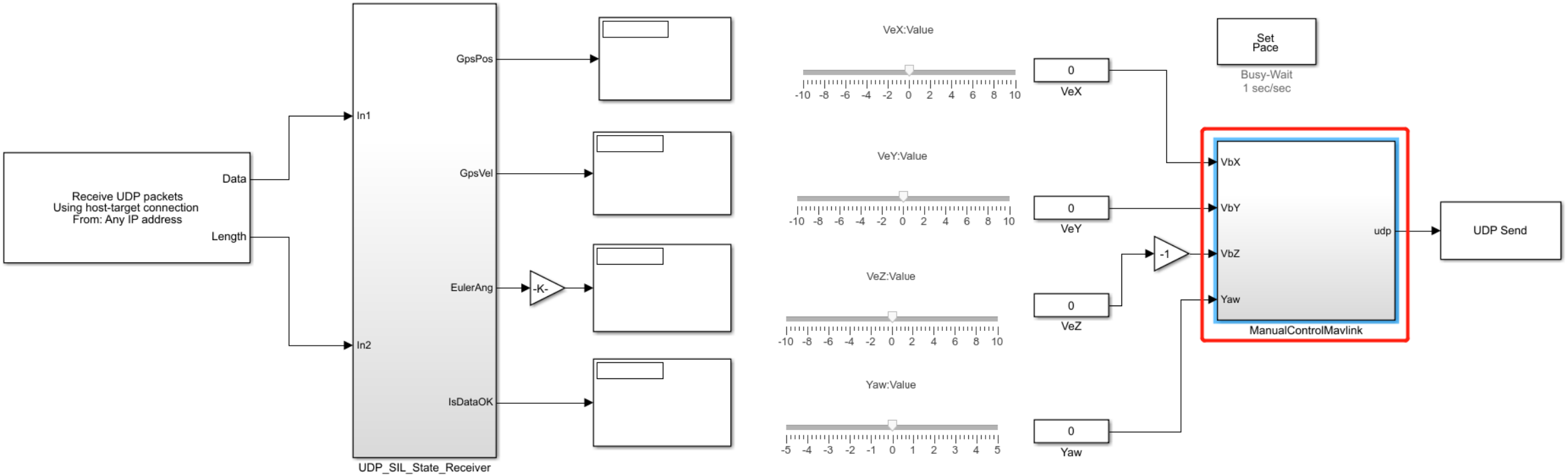
- 原理：接收CopterSim通过UDP发送的outHILStateData结构体的数据，并从中提取感兴趣的值
- CopterSim数据来源：转发来自Pixhawk的Mavlink消息，包括LOCAL_POSITION_NED，ATTITUDE，HOME_POSITIONE，STIMATOR_STATUS等

```
struct outHILStateData{ // mavlink data forward from Pixhawk
    uint32_t time_boot_ms; //Timestamp of the message
    uint32_t copterID;     //Copter ID start from 1
    int32_t GpsPos[3];     //Estimated GPS position，lat&long: deg*1e7, alt: m*1e3 and up is positive
    int32_t GpsVel[3];     //Estimated GPS velocity, NED, m/s*1e2->cm/s
    int32_t gpsHome[3];    //Home GPS position, lat&long: deg*1e7, alt: m*1e3 and up is positive
    int32_t relative_alt;  //alt: m*1e3 and up is positive
    int32_t hdg;           //Course angle, NED,deg*1000, 0~360
    int32_t satellites_visible; //GPS Raw data, sum of satellite
    int32_t fix_type;      //GPS Raw data, Fixed type, 3 for fixed (good precision)
    int32_t resrveInit;    //Int, reserve for the future use
    float AngEular[3];     //Estimated Euler angle, unit: rad/s
    float localPos[3];     //Estimated locoal position, NED, unit: m
    float localVel[3];     //Estimated locoal velocity, NED, unit: m/s
    float pos_horiz_accuracy;  //GPS horizontal accuracy, unit: m
    float pos_vert_accuracy; //GPS vertical accuracy, unit: m
    float resrveFloat;     //float,reserve for the future use
}
```

3.PX4官方控制器通信

3.2 Simulink模拟发送归一化遥控信号控制无人机

- 上一个例子发送的是遥控器的原始数据，因此PWM的输出值需要与遥控器校准值一致，否则可能出现控制偏差
- 打开“RflySimAPIs\SimulinkControlAPI\SimulinkControlAPI”，得到和上一个例子相同功能的demo，实验流程相同，但是本demo不需要关心遥控器校准值



3.PX4官方控制器通信

3.2 Simulink模拟发送归一化遥控信号控制无人机

- 该消息是通过实现MANUAL_CONTROL的mavlink消息，在实际飞行时，也可以通过数传发送该信号来复现控制指令。

MANUAL_CONTROL (#69)

This message provides an API for manually controlling the vehicle using standard joystick axes nomenclature, along with a joystick-like input device. Unused axes can be disabled an buttons are also transmit as boolean values of their

Field Name	Type	Description
target	uint8_t	The system to be controlled.
x	int16_t	X-axis, normalized to the range [-1000,1000]. A value of INT16_MAX indicates that this axis is invalid. Generally corresponds to forward(1000)-backward(-1000) movement on a joystick and the pitch of a vehicle.
y	int16_t	Y-axis, normalized to the range [-1000,1000]. A value of INT16_MAX indicates that this axis is invalid. Generally corresponds to left(-1000)-right(1000) movement on a joystick and the roll of a vehicle.
z	int16_t	Z-axis, normalized to the range [-1000,1000]. A value of INT16_MAX indicates that this axis is invalid. Generally corresponds to a separate slider movement with maximum being 1000 and minimum being -1000 on a joystick and the thrust of a vehicle. Positive values are positive thrust, negative values are negative thrust.
r	int16_t	R-axis, normalized to the range [-1000,1000]. A value of INT16_MAX indicates that this axis is invalid. Generally corresponds to a twisting of the joystick, with counter-clockwise being 1000 and clockwise being -1000, and the yaw of a vehicle.
buttons	uint16_t	A bitfield corresponding to the joystick buttons' current state, 1 for pressed, 0 for released. The lowest bit corresponds to Button 1.

3.PX4官方控制器通信

3.3 Simulink模拟发送使用Offboard模式的控制无人机 （推荐）

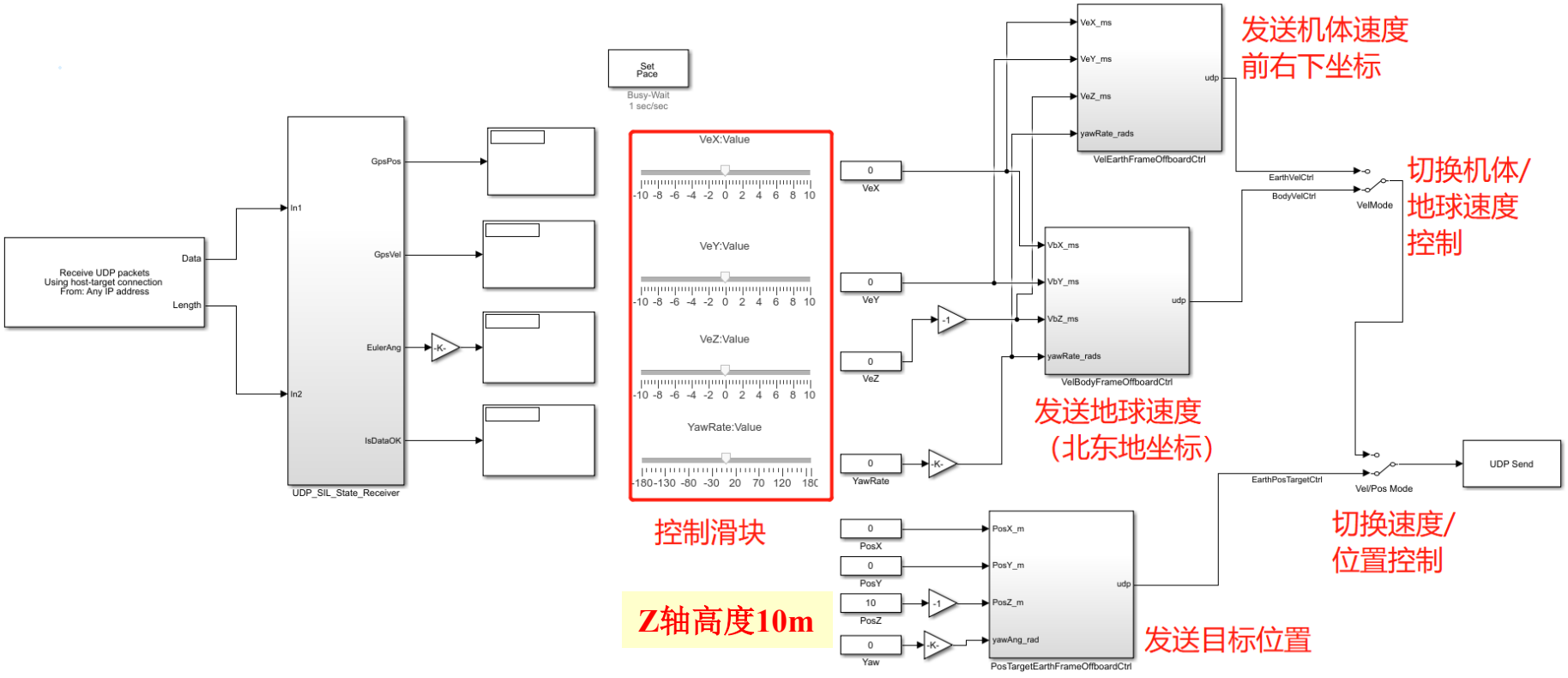
- Offboard模式是无人机的一种控制模式，通常给机载计算机或地面计算机（上位机）实时控制飞机的速度、位置、姿态等，可以把飞机当成一个整体对象，专注于顶层的视觉与集群算法开发。
- 遥控器信号控制没法定量地控制飞机的速度，因此使用起来没有Offboard控制模式方便，但是遥控器信号控制模式是最接近人的操作的方式，在一些高机动表演控制中有更好的效果

- 本课程后续实验都是将飞机看出一个整体子对象（接收并实现速度/位置/加速度/航路等指令），因此后续系列实验将主要采用Offboard模式来控制飞机。由于Offboard模式是PX4官方控制器提供的功能，因此需要确保Pixhawk运行的是官方固件（在前面的设置中有提到）。

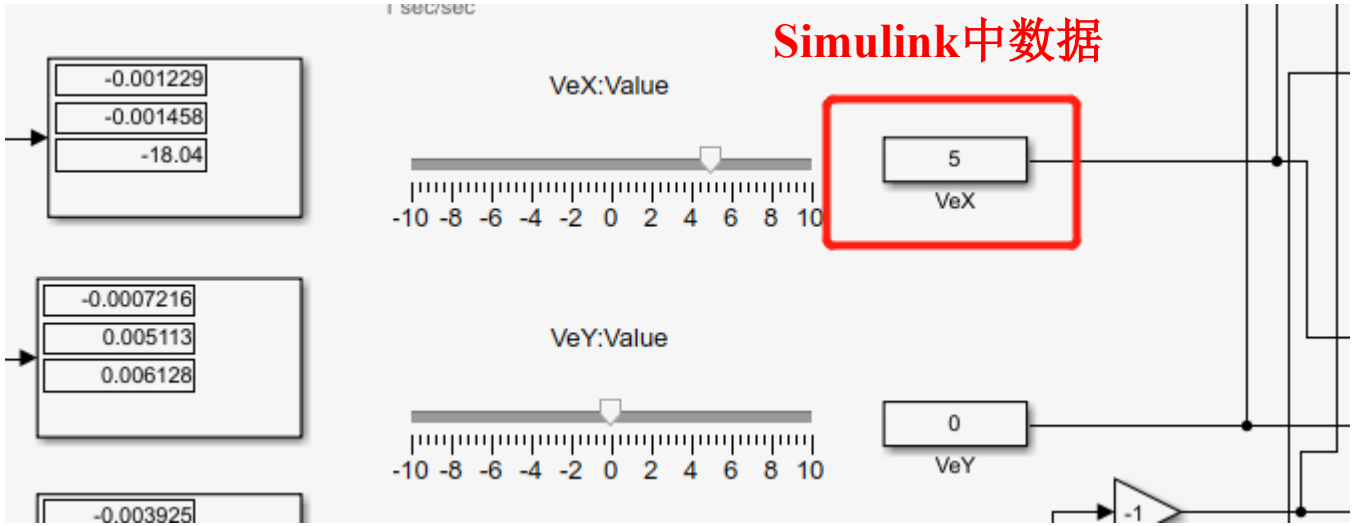
3.PX4官方控制器通信

3.3 Simulink模拟发送使用Offboard模式的控制无人机（推荐）

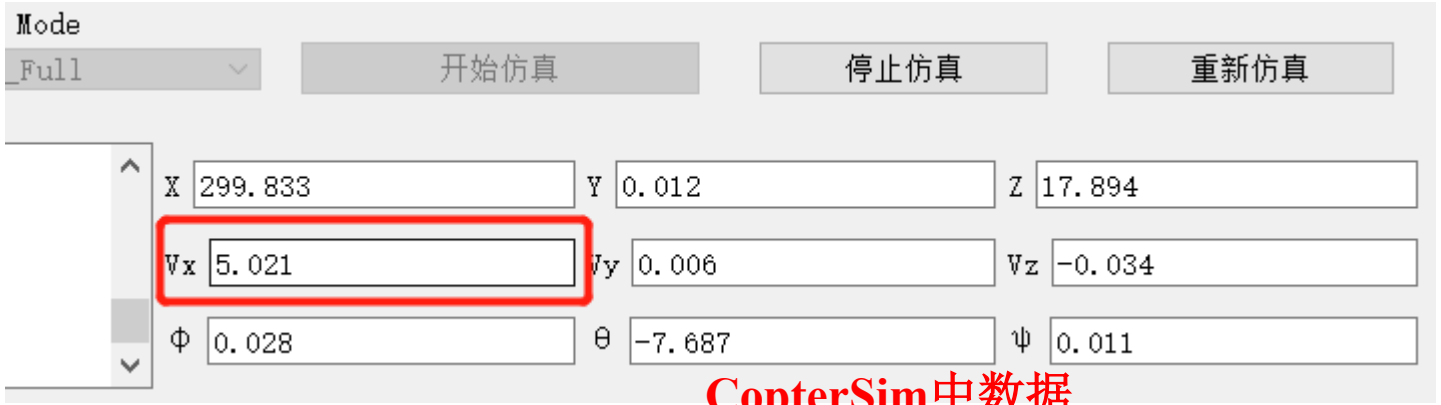
- 开启CopterSim的硬件在环（或软件在环）仿真系统
- 打开“RflySimAPIs\SimulinkControlAPI\OffboardAPI.slx”运行即可，可以看到飞机首先自动起飞到10m高度，然后切换“速度/位置控制”开关，拖动滑块，可以输入X方向速度5（或拖动滑块到期望值），在QGC中观察速度与给定速度是否一致



3.PX4官方控制器通信









3.PX4官方控制器通信

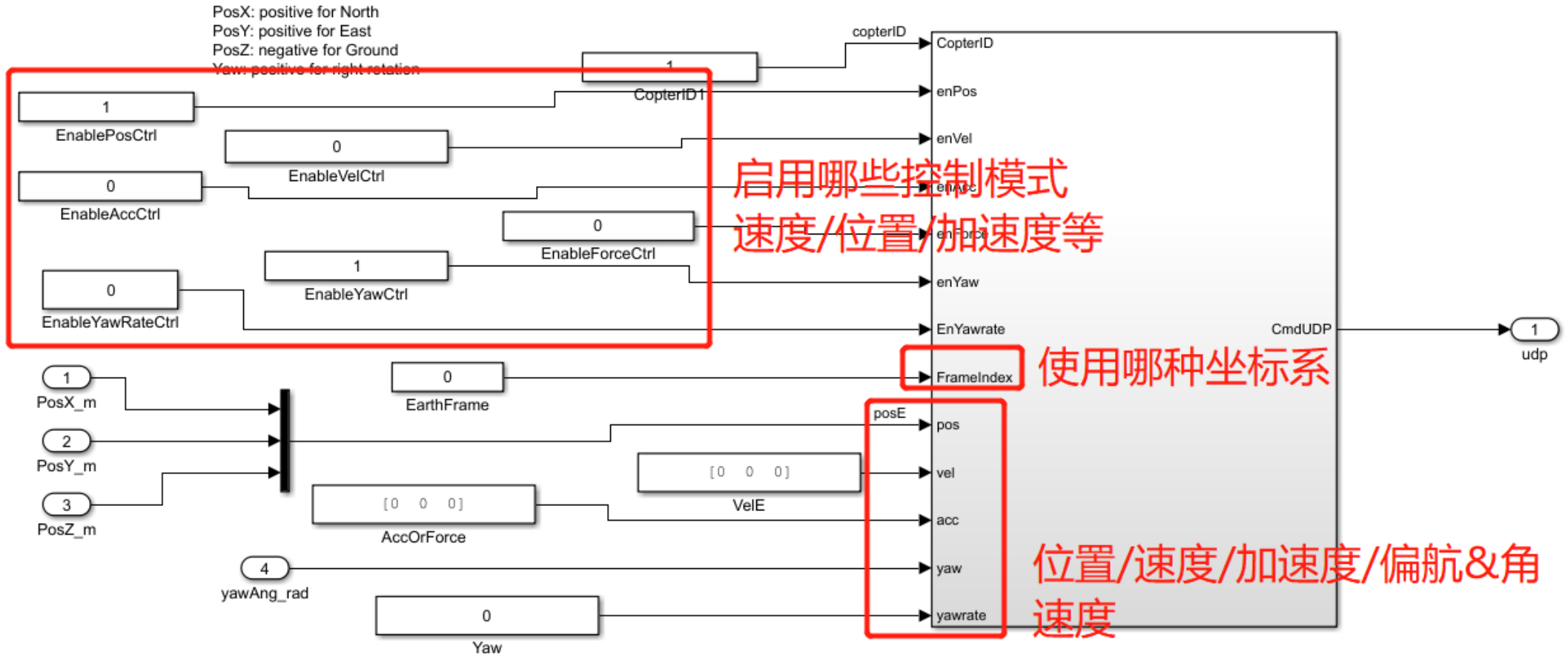
3.3 Simulink模拟发送使用Offboard模式的控制无人机（推荐）

- 原理：该例子会使PX4进入Offboard模式，然后发送SET_POSITION_TARGET_LOCAL_NED的mavlink消息来实现飞机的速度、位置、与角度等控制。该指令不需要飞机进行遥控器校准或模态设置，直接给定指定的速度或位置即可。
- 在OffboardAPI.slx文件中展示了三个模块例子，VelEarthFrameOffboardCtrl是地球坐标系下速度控制模块，VelBodyFrameOffboardCtrl是机体坐标系下速度控制，PosTargetEarthFrameOffboardCtrl是地球坐标系下位置控制模块（给定相对起飞点的x，y，z坐标，飞机会自动飞到该点并悬停）。
- 三个模块实现方法完全相同，只不过是Offboard消息的几个参数（位置/速度控制模式&机体/地球坐标系）不同，Offboard控制指令是以解锁的位置为Home_Position坐标为原点的相对坐标（Local_Position），因此发送的位置指令是指飞到相对解锁位置的相对坐标值。

3.PX4官方控制器通信

3.3 Simulink模拟发送使用Offboard模式的控制无人机 （推荐）

- Simulink的Offboard模式接口如下图，可以自行组合需要控制的指令



3.PX4官方控制器通信

3.3 Simulink模拟发送使用Offboard模式的控制无人机 （推荐）

- https://mavlink.io/en/messages/common.html#SET_POSITION_TARGET_LOCAL_NED MAVLink消息的实现方法如下图

SET_POSITION_TARGET_LOCAL_NED (#84)

[Message] Sets a desired vehicle position in a local north-east-down coordinate frame. Used by an external controller to command the vehicle (manual controller or other system).

Field Name	Type	Units	Values	Description
time_boot_ms	uint32_t	ms		Timestamp (time since system boot).
target_system	uint8_t			System ID
target_component	uint8_t			Component ID
coordinate_frame	uint8_t		MAV_FRAME	Valid options are: MAV_FRAME_LOCAL_NED = 1, MAV_FRAME_LOCAL_OFFSET_NED = 7, MAV_FRAME_BODY_NED 8, MAV_FRAME_BODY_OFFSET_NED = 9
type_mask	uint16_t		POSITION_TARGET_TYPEMASK	Bitmap to indicate which dimensions should be ignored by the vehicle.

3.PX4官方控制器通信

x	float	m		X Position in NED frame
y	float	m		Y Position in NED frame
z	float	m		Z Position in NED frame (note, altitude is negative in NED)
vx	float	m/s		X velocity in NED frame
vy	float	m/s		Y velocity in NED frame
vz	float	m/s		Z velocity in NED frame
afx	float	m/s/s		X acceleration or force (if bit 10 of type_mask is set) in NED frame in meter / s^2 or N
afy	float	m/s/s		Y acceleration or force (if bit 10 of type_mask is set) in NED frame in meter / s^2 or N
afz	float	m/s/s		Z acceleration or force (if bit 10 of type_mask is set) in NED frame in meter / s^2 or N
yaw	float	rad		yaw setpoint
yaw_rate	float	rad/s		yaw rate setpoint

大纲

1.软硬件配置

2.MAVLink通信解析

3.PX4官方控制器通信

4.代码生成控制器通信

5.小结

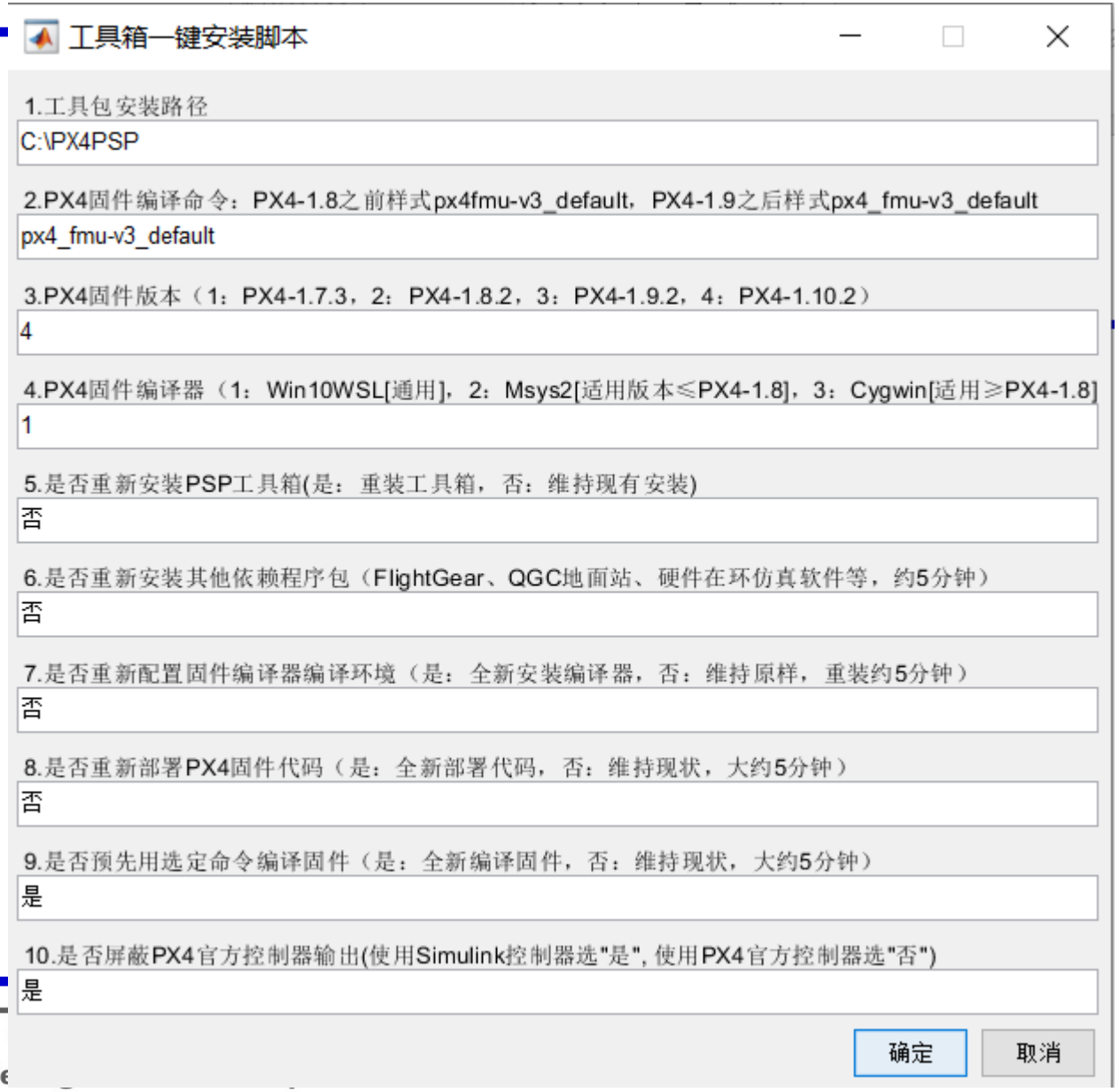
本小节的例程源码路径：
RflySimAPIs\SimulinkControlAPI\Rfly_API_CTRL

4.代码生成控制器通信

注：本小节内容主要针对课程“RflySim高级版_第02讲_飞行控制算法开发”（即《多旋翼飞行器设计与控制实践》书中的系列实验）中用Simulink开发的控制器的外部通信问题。

4.1 自动代码生成环境配置

- 针对Simulink生成的Pixhawk控制算法也可以通过的QGC/Simulink API进行控制。运行本小节的例子，需要使用Simulink代码生成控制器，因此需要重新运行安装脚本，如右图所示配置（RflySim高级版）来屏蔽PX4自身输出。注： RflySim基础版请使用px4fmu-v3_default, 1.7.3固件，和Msys2编译器
- 注意：编译命令需要根据自己硬件修改
- 注意：Simulink自动生成的代码目前不支持px4_sitl_default的编译命令，因此不支持PX4 SITL仿真，本小节例子需要使用Pixhawk硬件进行硬件在环仿真。

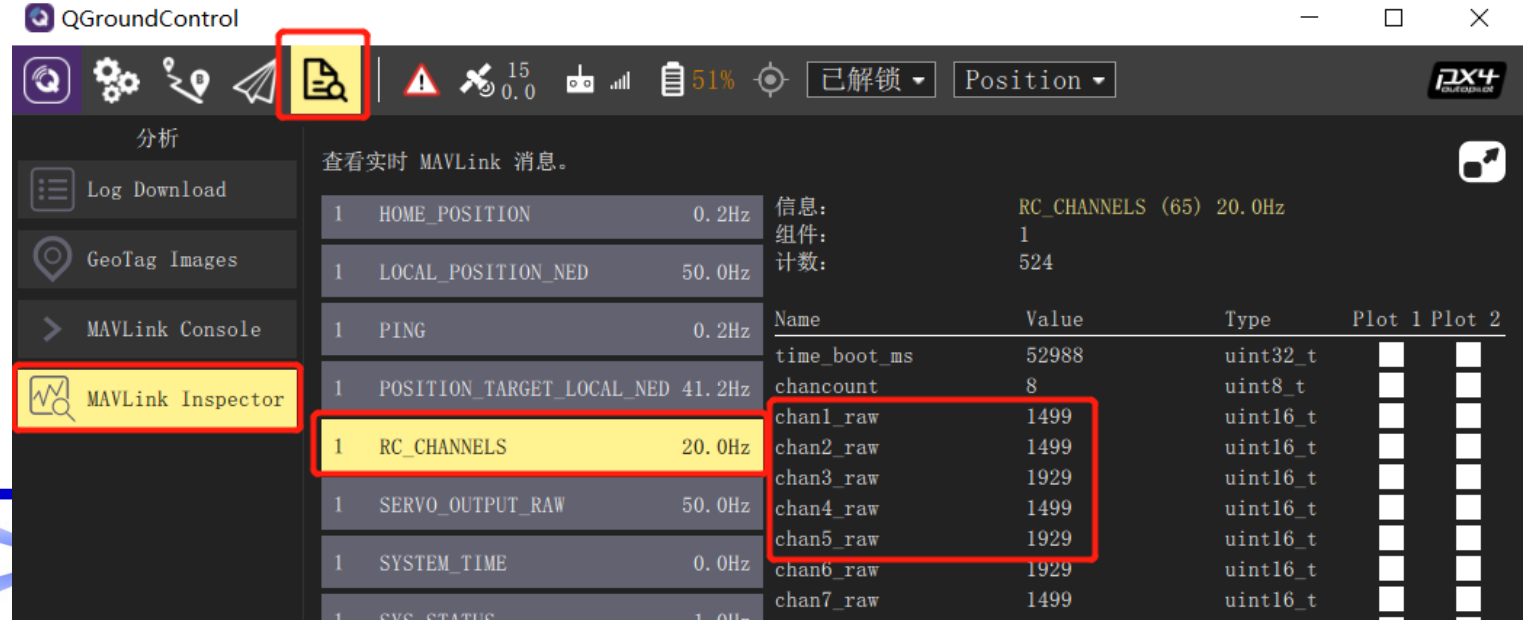


北航可靠
BUAA Reliable

4.代码生成控制器通信

4.2 使用Simulink生成的遥控器信号控制Simulink设计的控制器

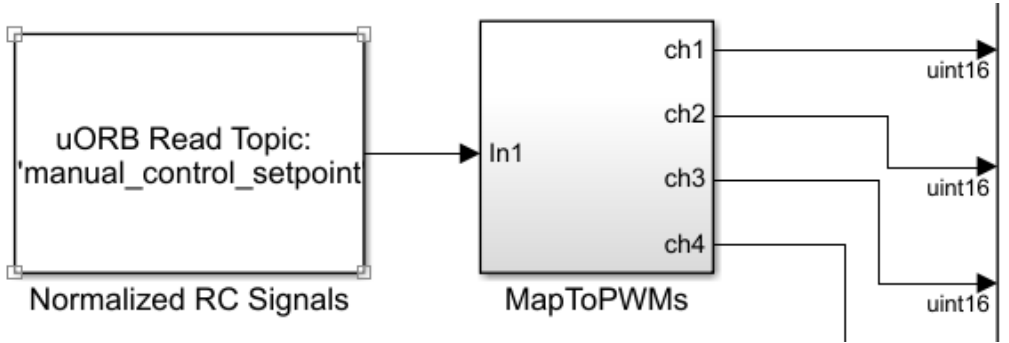
- 可以使用Simulink输出的遥控器信号控制我们自己设计的姿态控制器，例如“RflySimAPIs\FlightControlExpCourse\code\e0\3.DesignExps\Exp4_AttitudeSystemCodeGen_old.slx”
- 首先打开“Exp4_AttitudeSystemCodeGen_old.slx”编译固件并烧录Pixhawk，然后用CopterSim连接Pixhawk并开始硬件在环仿真（高级版可以直接运行桌面HITLRun一键快速打开硬件在环）。
- 打开并运行“RadioControlAPI.slx”文件，可以控制飞机起飞并前后左右移动。在QGC中也可以观察到数据



4.代码生成控制器通信

4.2 使用Simulink生成的遥控器信号控制Simulink设计的控制器

- 原理：进一步解释原理，RadioControlAPI.slx文件发送RC_CHANNELS_OVERRIDE的mavlink消息到Pixhawk，而Exp4_AttitudeSystemCodeGen_old.slx中使用的遥控器模块会接收该Mavlink消息，因而可以响应。
- 同理，如果需要自己生成的代码能响应ManulControlAPI.slx和OffboardAPI.slx中的输入，需要分别在Simulink控制器中接收相应的uORB消息。其中ManulControlAPI.slx对应的是“MANUAL_CONTROL_SETPOINT”的uORB消息，OffboardAPI.slx对应的是“POSITION_SETPOINT”的uORB消息。
- 注：ManulControlAPI.slx对应了“Exp4_AttitudeSystemCodeGen.slx”代码生成的例程，可以响应“MANUAL_CONTROL_SETPOINT”消息。





4.代码生成控制器通信

4.3 使用Simulink收发自定义消息

- RflySim平台还提供了一个外界程序与自己Simulink控制器进行交流的消息。它发送rfly_ctrl.msg（见Firmware\msg文件夹）的uORB消息。定义如下：

```
uint64 timestamp                    # time since system start (microseconds)
uint64 flags                        # control flag
uint8 modes                 # mode flag
float32[16] controls                # 16D control signals
```

- 该uORB消息可以通过外部程序发送mvlink消息来实现“HIL_ACTUATOR_CONTROLS”消息，其16维的controls控制量对应于下面的定义。

HIL_ACTUATOR_CONTROLS (#93)

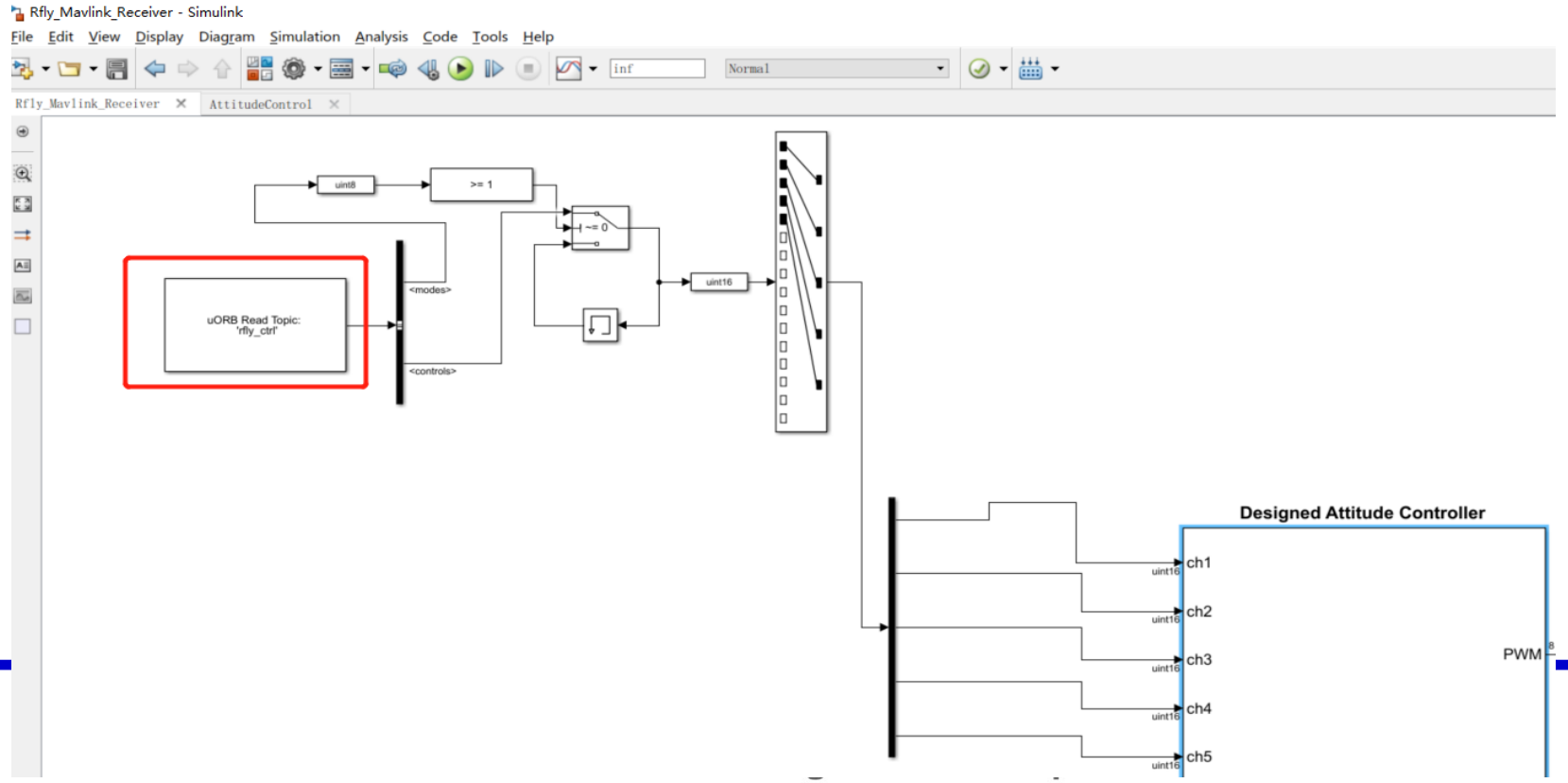
Sent from autopilot to simulation. Hardware in the loop control outputs (replacement for HIL_CONTROLS)

Field Name	Type	Units	Values	Description
time_usec	uint64_t	us		Timestamp (UNIX Epoch time or time since system boot). The receiving end can infer timestamp format (since 1.1.1970 or since system boot) by checking for the magnitude the number.
controls	float[16]			Control outputs -1 .. 1. Channel assignment depends on the simulated hardware.
mode	uint8_t		MAV_MODE_FLAG	System mode. Includes arming state.
flags	uint64_t			Flags as bitfield, reserved for future use.

4.代码生成控制器通信

4.3 使用Simulink收发自定义消息

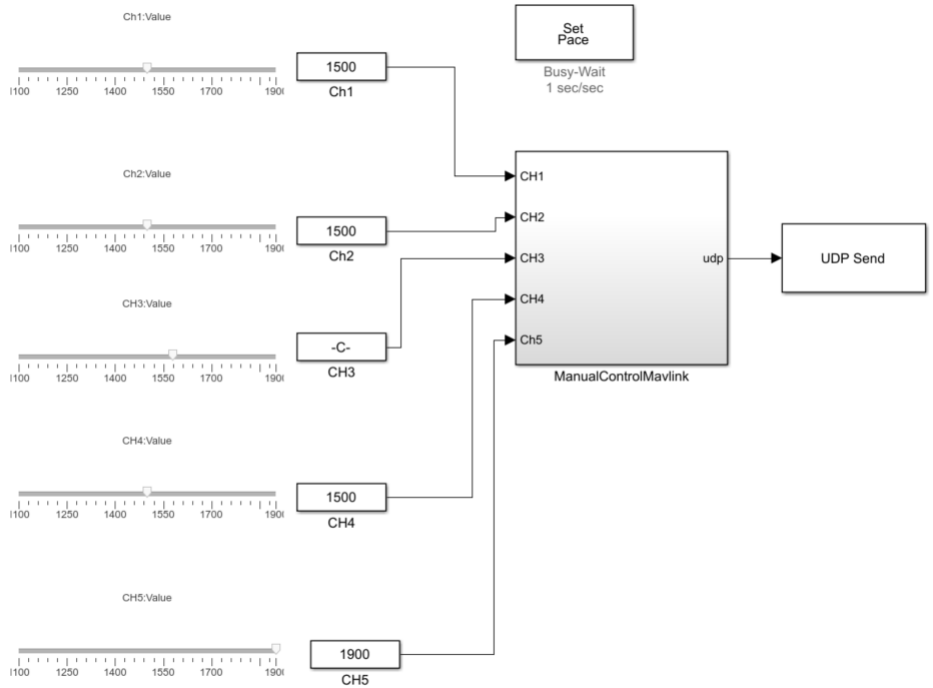
- 用MATLAB打开RflySimAPIs\SimulinkControlAPI\Rfly_API_CTRL\Rfly_Mavlink_Receiver.slx，编译生成代码，并烧录到Pixhawk中。如下图所示，它接收了rfly_ctrl的uORB消息，替换了原来的遥控器信号。



4.代码生成控制器通信

4.3 使用Simulink收发自定义消息

- 开启CopterSim和Pixhawk的硬件在环仿真，运行“Rfly_API_CTRL\Rfly_Mavlink_API_Sender.slx”文件，该文件可以向CopterSim发送控制信号，CopterSim会转发为“HIL_ACTUATOR_CONTROLS” Mavlink消息到Pixhawk，然后Pixhawk再发布到rfly_ctrl的uORB消息池中，给上图中代码生成的px4_simulink_app使用。如下图所示，该slx例程模拟了CH1~Ch5通道的遥控器数据，发送到了rfly_ctrl消息中。
- 本例程的实验效果与3.1中模拟遥控器信号控制PX4官方控制器的操作流程一致
- 实验现象与4.2中发送遥控器信号到Simulink代码生成控制器的效果相同，因为两个例子都是发送了遥控器信号，只不过通信途径有区别。
- 注：在真机飞行是，用MAVLink向飞控发送HIL_ACTUATOR_CONTROLS消息即可，本模块可以实现向控制发送数据



谢谢！